REST API

PUBLIC FOR TALGIL SERVERS

v1.47

February 25, 2026

1 Document Control

Version	Date	Description
1.0	6 Sep 2015	Deprecated cookie authentication, use custom header and API key instead
...	...	...
1.16	3 Jun 2018	Added /mytargets request
1.18	3 Jul 2018	Added info/setting/params controller sections
1.19	13 Sep 18	Added batch consumption query requests
1.20	12 Nov 18	Fixed typos in URLs of consumption queries
1.21	20 Jan 2019	Added program management calls, virtual sensors updates
1.22	09 Aug 2019	Added support for modification of fertilizer dosage in sequences
1.23	20 Aug 2019	Corrected alarm object's structure, added alarm types appendix
1.24	22 Oct 2019	Added raw data queries for line objects, central and local fert and filter sites
1.25	14 Nov 2019	Added batch queries for fertilizer consumptions
1.26	25 Nov 2019	Added access to objects by uid
1.27	16 Dec 2019	Added queries for system events
1.28	23 Feb 2020	Typo in import virtual sensor updates
1.29	07 Apr 2020	Added analysis library calls
1.30	16 Apr 2020	Added fertilizer recipe calls
1.31	24 Jun 2020	Minor refactoring of explanations, examples
1.32	09 Jul 2020	Updated batch fert consumptions requests
1.33	15 Oct 2020	Minor refactoring, fixing the object references format, added batch observations query.
1.34	03 Nov 2020	Added GIS section, corrections in fert site related objects
1.35	16 Feb 2021	API Key generation policy changed, Extended raw data queries with more observation types
1.36	18 Feb 2021	Added 'volume' option for consumption queries
1.37	03 Mar 2021	Fixed a typo in program start 'from' parameter

1.38	05 Apr 2021	- water batch consumptions valves parameter is optional - extended format to allow IDD indexes - fert batch consumptions call will allow valves parameter
1.39	04 May 2021	- added analog sensor properties - batch consumptions output format will include uid
1.40	29 September 2021	- Changed screenshots and text referring to API-key - Corrected and updated field description
1.41	31 May 2022	- Corrected and updated field description
1.42	1 July 2023	- Added missed fields in Params section - Added new modifications options for Valves, Programs, Conditions and 1st Container - Added possibility to use parameters names in modifications requests - Added new filters options to the GET requests - Extended errors and exceptions messages with additional details. - Updated Analog sensor types and units - Corrected and updated field descriptions
1.43	25 February 2024	- Added “lastStopTime” parameter for the program - Added parameters "additionalBefore", "diffMainValveDelay" and "userDefinedId" to the 1st Container - Adjusted parameters order in some containers - Added new program states "Running before reset" and "Waiting before reset" values to the parameter description
1.44	5 May 2024	- Full image request syntax update. `/targets/{id}/`
1.45	14 February 2025	- Added sensors types 45 and 46 - Added sensor units 51, 52 and 53 - Added “subType” to the Info - Added “boosterCloseDelay” to the Params - Added “waterBeforeAdditional” to the Programs - Added Chapter 6.16.2 “Plugin” - Modified Chapter 6.16.1 “RTU” - Added plugin addresses to all inputs and outputs elements - Added Chapter 6.25 “Analog values” - Made diverse texts and layouts corrections in the document
1.46	14 July 2025	- Added parameter “isAreaExtendedReady” - Added parameter “sensor” to Contacts - Added separate endpoint to request water consumption from the Virtual Water Meters.

1.47	25 February 2026	- Added parameter “supportsNetIDs” to Params - Added parameter “mbu” to appropriate elements Analog Sensors, Analog Values, Satellites, Contacts. - Added parameter “rssi” to the Status - Added sensor types 47 - 50 - Added sensor units 54 – 57 - Added interface types 17 and 18 - Added parameter “netID” to the Interface - Added calculations types 7, 8 and 9 for the Analog Sensor - Added plugin card types 6 and 7 - Added controller alarm types 36 and 37 - Added object types containers 49 – 52 - Added new reaction type of the fertilizer site 4 - Updated section 3.8 to the actual procedure status.

2 Document Identification

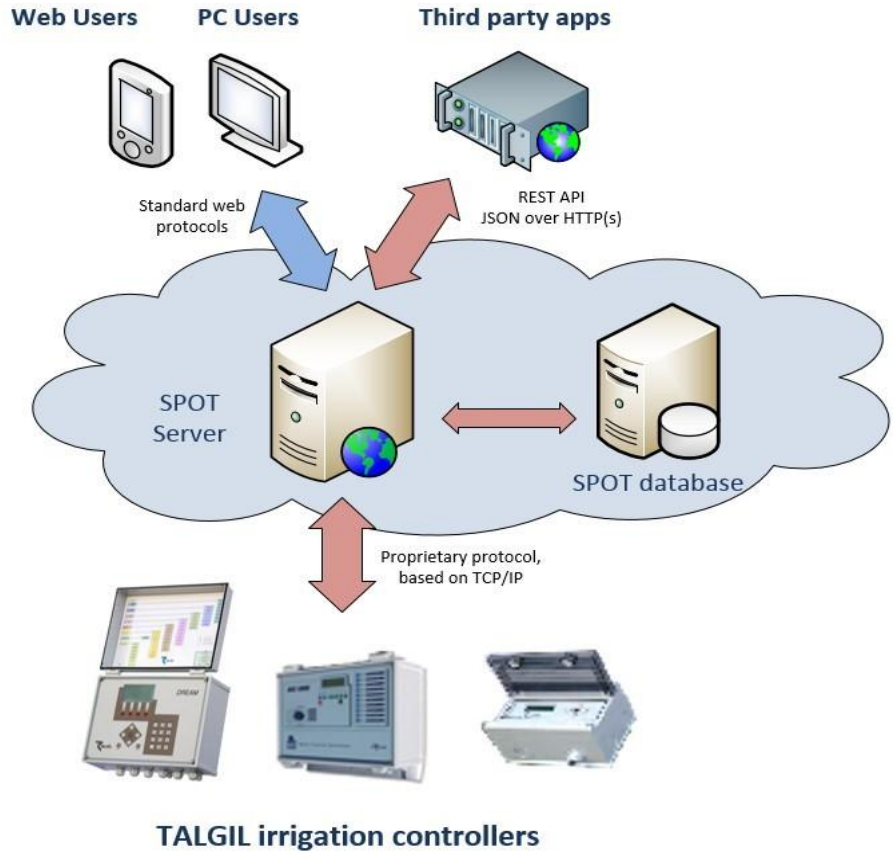
The purpose of this document is to describe programming interface for TALGIL SPOT servers. Using the API developers will be able to communicate directly with irrigation controllers by reading their real-time states, issuing commands, analysing historical data and more.

3 Introduction

The API described in this document offers third party developers simply means of integrating their applications with TALGIL SPOT servers. The top right corner of the diagram below shows where this document is focusing.

The API follows widely accepted industry standards and is platform neutral. It means that any kind of third party application should be able to work with SPOT servers as long as they follow the formats described in this document.

The API is based on **REST** paradigm. The data sent to and received from servers is in **JSON** format. The protocol used for the communication between applications and SPOT servers is secured **HTTP**. Subsequent sections in the document describe in details requests, responses and data structures used.



3.1 Server account

In order to start using SPOT servers API, the developer is required to have an active user account and **API access key**. Please contact us for more details.

Every server call must be accompanied with the HTTP header named **TLG-API-Key** and the value of the **API access key** obtained for the user account. Please note that if user name or password are modified, the API Key must be re-issued.

Note that access to server resources is strictly limited by the features and permissions applied for the user account. For example, the caller will be permitted to access data only on controllers available for this account.

3.2 URL formats

The URL format of all requests is following the same simple rule. Throughout this document we will be referring only to the part which follows this:

```
https://srv.talgil.com/api/
```

The subsequent part of the URL will always indicate what resource should be referenced in request. The type of request will indicate what type of operation is being performed on resource. Note that URL mappings on server side are case sensitive and in most cases are lower case characters.

For example, the following request is dedicated to obtain a state of irrigation valve 5 located in irrigation line 2 of controller with serial number 12345:

```
GET /targets/12345/lines/2/valves/5/state
```

3.3 HTTP methods

SPOT API requests are made using standard HTTP methods, each one carrying a specific connotation:

GET	Considered a "safe" method as any requests to the API with an HTTP GET are non-volatile and will not change any values of underlying resource
POST	Generally used for executing commands, creating objects and sending data to server

3.4 HTTP headers

Every request must have TLG-API-Key header with a valid API key generated from user account which is used for authentication and authorization of the caller, for example:

```
TLG-API-Key: 1sw61svs1t3b1v9g1aif
```

Some requests may require data to be sent (or posted) to server. The format of this data is always JSON throughout the API and is described in details whenever required. The caller will specify corresponding header to indicate content type like this:

```
Content-Type: application/json
```

3.5 Request parameters

Some requests may include optional parameters to the call. This will also be specified explicitly when needed. For example, the following call tries to obtain historical trend of valve 5 in line 2 of controller 123 in the time range from x to y.

```
GET target/123/lines/2/valves/5/trend?from=x&until=y
```

3.6 Response format

If the response from the API contains a body, it will always be in JSON format which is described specifically for each data type throughout this document.

3.7 HTTP status codes

200 OK	Indicates that the request was successful
400 Bad Request	The request body was malformed. Accompanied by a list of errors in the response body.
401 Unauthorized	Indicates that request for unauthorized resource was made
404 Not Found	The resource could not be found
405 Method Not Allowed	The method you are using to access (GET, POST, PUT, etc.), is not allowed for the resource.
415 Unsupported Media Type	Media type is unsupported. Most resources are available as JSON only, though a few have alternate types as described in this API documentation.
500 Internal Server Error	There has been an error from which server could not recover. There is a high likelihood that an internal server error represents a bug on the part of the server.

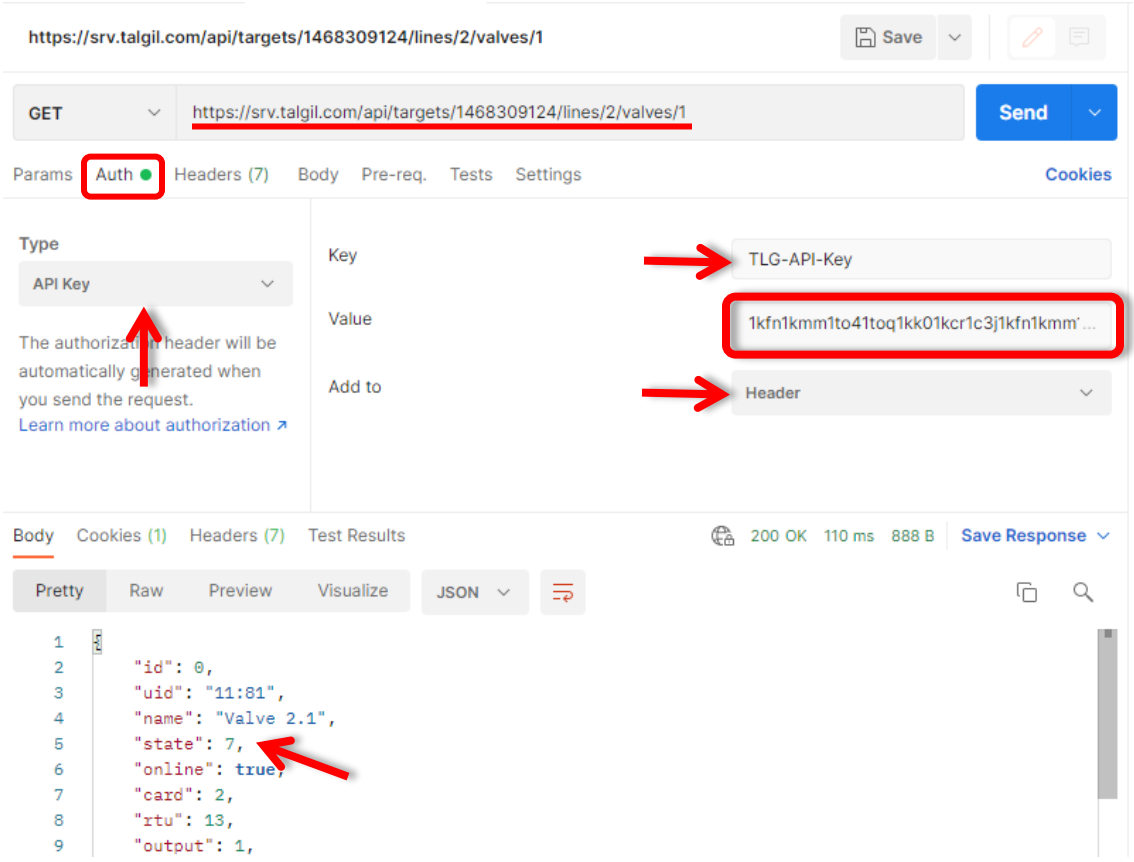
3.8 Environment setup

This section gives step by step instructions on how to get started with the API by using conventional browser and the Postman application as a sample

Note that for the remote calls our user (**rest**) needs to have **DESKTOP** access type and it must belong to the same project as our controller. The next step is to get the API key, please contact us via **Talgil general mail** (Send mail) to obtain one and get more information about our API.

3.8.1 Client Application

For making calls to our server, in this tutorial we will use an app named Postman. It is a nice and simple REST client. You can of course use any other client which provides similar functionality. Once Postman is started, go to **Authorization** tab and choose in 'Type' the option **API key** this will be needed for the **API authentication**. You need to enter the following values into following the fields: in 'key' field enter **TLG-API-Key**, in 'Value' field paste the API key which you will get from our support team. 'Add to' field choose **Header**. This will automatically authenticate all the calls we will be doing from this client to the server.



At this point we are ready to call the server by entering the URL at the top. In this example, our server is located at **srv.talgil.com** and working over encrypted channel, note the **https**. We are calling for the data of "***first*** *irrigation valve positioned in the* ***second*** *line of controller with serial number* ***1468309124*****"**. The response indicates that this valve is in LOW_FLOW at this moment (state 7) and there is a bunch of other useful info in JSON response.

The rest of this document describes how to construct the URL part of the call-in order to obtain needed information from controllers as well as how to send commands and interpret responses.

4 Administration calls

4.1 List controllers

The following call will provide a list of controllers (targets) linked to the caller account.

URI	GET /mytargets	
Parameters	user	*optional, when specified the response will include targets belonging to **both** caller and 'user' accounts. Intended for use by proxy users only.*
Response JSON	[{	*array of time-value pairs*
	serial: number	*controller serial number*
	name: string	*controller name*
	affiliate: string	*controller affiliate name*
	project: string	*controller project name*
	online: boolean	*true when controller is currently connected to server*
	},...]	

Examples	GET /mytargets [{ serial: 33456456, name: “foo”, affiliate: “company name”, project: “banana”, online: true },...]	*list of targets belonging to the caller only*
	GET /mytargets?user=xxx [{ serial:56789767, name: “foo”, ... },...]	*list of targets belonging to the caller **and** xxx user*

4.2 Automatic target proxifying/unproxyfying

This request will be accessible only for approved API partners. It allows to get access to the controller of the client without contacting Talgil support.

This request will need the same data as our “Permission to API access” form and basically is the electronic version of it in a form of the request.

If you are interested and would like to know more - please contact us

5 Controller Image

Each irrigation controller supported by this API represents a data structure comprised of logical objects which are mapped directly to the physical structure of the remote physical controller. This data structure is referred to as **controller image**, or just **image** throughout this document.

The image is a tree like structure where objects are listed and nested in a tree-like form with the controller itself as a root node. Different controller types have different structure of images, but they all are tree-like structures which can be browsed with this API.

There are few ways to obtain parts of the controller image:

1. URL mapping to a specific object (or objects by type)
2. URL mapping to objects by their unique id's (**uid**) (***OBSOLETE***)

Using either of the methods requires some knowledge and understanding of the underlying controller. For example, in order to get an access to a particular valve, the caller will either seek for all valves and then pick needed object from the list, or use valve **uid** to get its structure directly.

Here, the reference to a **first valve in first line** can be done either by doing this:
`GET targets/123/lines/1/valves/1`

or, by using the **uid** obtained from previous calls, where **11.1** is the first valve in first line:
`GET targets/123/idd/11/1` (***OBSOLETE***)

Mapping of the image objects to URL's are described in corresponding sections for each particular type of controller and type of object.

6 Dream controllers

URI	GET /targets/{id}/	
Info	Gets an entire image structure of controller(s). Should be used at least once per application to determine controller structure. When {id} is specified, the server will respond with contain structure of this specific controller where {id} is the serial number. When {id} is not specified, the response will contain list of all controllers currently assigned to the caller.	
Params	filter	serial - include only serial number in response state - include only controller state in response
Response	dream: {	
	info: array	*controller identification info*
	state: array	*global controller states*
	params: array	*controller setting and parameters*
	battery: array	*reference to battery object*
	alarms: array	*list of outstanding alarms*
	mainValves: array	*collection of main valves, optional*
	sources: array	*collection of water sources*
	vwms: array	*collection of virtual water meters*
	conditions: array	*collection of condition objects*
	sensors: array	*collection of sensor objects*
	avalues: array	*collection of analog values objects*
	sensorSites: array	
	contacts: array	*collection of contact objects*
	satellites: array	*collection of satellite objects*
	aouts: array	
	interfaces: array	*collection of interface card objects*
	lines: array	*collection of irrigation lines*
	groups: array	
	programs: array	*collection of program objects*
	filterSites: array	*collection of central filter sites*

	fertSites: array	*collection of central fert sites*
	wms: array	*collection of free water meters*
	weather: object	*Weather station if present*
	}	

6.1 Info

This section contains properties describing controller in terms of type, version and other useful bits of information.

Property	Type	Description
serial	number	Controller serial number generated during manufacturing
name	string	Controller name, user specified
app	enum	Type of application running by the controller 1 - DREAM 2 - OASIS 4 – SAPIR
subtype	enum	Device Sub Type for Sapir2 0 – SAPIR2 1 – SAPIR MINI
version	string	Firmware version
tzOffset	number	Time zone offset in minutes
metric	enum	Metric system this controller is using: 0 - Metric system 1 - Imperial system (US etc)

6.2 State

This section holds real-time state of controller and its main modules.

Property	Type	Description
online	boolean	Indicates whether the controller is currently available for direct communication
time	number	Current local time (UTC ms)
configTime	number	The time when image was modified, UTC ms.
resetTime	number	The time when controller software was reset (minutes since nearest midnight), UTC ms.
alarms	boolean	Has outstanding alarms
irrigation	enum	State in terms of irrigation 0 - unknown 1 - idle 2 - idle with problems 3 - irrigating 4 - irrigating with problems 5 - frozen
flushing	enum	State in terms of flushing 0 - unknown 1 - not connected 2 - idle 3 - flushing
comm	enum	State of communication (RTU, RF card, etc) 0 - unknown 1 - not connected 2 - ok 3 - error 4 - low battery
hardware	enum	Consolidated state of interface cards 0 - unknown 1 - ok 2 - error
ac	enum	State of controller AC (if present) 0 - unknown 1 - ok 2 - power fail
modem	enum	Modem state (if present)
rssi	number	Signal strength value if the controller connected with the Cellular of Wi-Fi modem
flow	boolean	Has flow of any kind

freezeReason	enum	When controller is frozen, this property indicates the reason 0 - manually by user request 1 - low battery 2 - ac power fail
today	number	Current irrigation day

*Blue marked fields are modifiable.

6.3 Params

This section holds options, definitions, defaults and other global controller settings.

Property	Type	Description
areaDosage	bool	Dosage per area option
lineFillDpControl	bool	DP control during line fill delay option
graduatedOpenDelay	number	Graduated opening delay option (in sec. 0=off)
agitatorMode	enum	agitators operation mode 0 – bulk 1 – periodically 2 – one-short mode 3 – sequential mode
boosterMode	enum	Global booster mode **(Dream only)** 0 - continuous 1 - alternating
boosterCloseDelay	number	Delay by the closing of the booster **(Dream only)**
agitatorOnTime	number	Agitator 'on' time in seconds, 0-3599
agitatorOffTime	number	Agitator 'off' time in seconds, 0-3599
noPressureDelay	number	No pressure delay in seconds, 0-3599
fertLeakageLimit	number	Fertilizer leakage limit, 0-99
areaUnits	enum	Default Area units for all valves 0 - dunam (or 100 ft2 with usa units) 1 - hectare 2 - acre
waterAccUnits	enum	when 'info.metric' = 1 (imperial) 0 - THG (thousand gallons) 1 - acre feet2 - acre inch when 'info.metric' = 0 (metric) 0 - cubic meters
runListSize	number	Number of days in the irrigation run list in days
stopTime	number	Stop time of all programs in controller, number of minutes since midnight (0 - 1440)

priorities	boolean	Enable prioritizing irrigation programs. In case of conflict, the program with the higher priority will irrigate and the other will wait. Higher number indicates higher priority.
groups	boolean	Allow usage of named groups like G1, G2, etc **(Dream only)**
fertSets	boolean	Allow using fertilization libraries
fertLimits	boolean	Allow using global fertilization limits
startTogether	boolean	Allow + in program sequences **(Dream only)**
workTogether	boolean	Allow & in program sequences **(Dream only)**
reuseValves	boolean	Allow the same valve to be used more than once in program sequence
specialBefore	boolean	Special amount of water before fertilization for the first fertilizer of each local fert. site is enabled **(Dream only)**
additionalBefore	boolean	Special amount of water before fertilization for the second fertilizer of each local fert. site is enabled **(Dream only)**
datalogHysteresis	number	Specifies how large (%) should be the change in order to be collected. Can be specified in fractions. Zero value forces collection of ALL data without tracing the changes.
maxPrograms	number	Maximum number of irrigation programs allowed to be created
cycles	boolean	Usage of program cycles enabled
evapoControl	boolean	Evaporation control is enabled
et[16]	array	Daily ET data, first item in array is yesterday's ET, second – the day before yesterdays, etc. Value in 0-999
parallelPrograms	boolean	Running parallel programs is allowed
conditions	boolean	Conditions are enabled **(Dream only)**
accumulatedRadiation	boolean	Use of accumulated radiation is enabled
frostProtection	boolean	Frost protection mechanism is enabled
rainDelay	boolean	Rain delay mechanism is enabled
sound	boolean	On-board sound is enabled **(Dream only)**
showIOCommErrors	boolean	Show IO communication errors

valveCtrlDelay	number	Delay for checking valve status (seconds)
nomFlowDefault	number	Default nominal flow for all valves (1-9999)
minFlowDeviation	number	Minimal flow deviation from nominal for all valves (%)
maxFlowDeviation	number	Default max flow deviation from nominal for all valves (%)
fillTime	number	Default fill up delay in minutes for all valves
leftFertEvent	number	Option of stop with left fert system event (no=0, yes=1) **(Dream only)**
fillPressControl	number	Enable pressure control during line fill delay (no=0, yes=1)
waterDosageMode	enum	Default water dosage mode 0 – hh:mm:ss 1 – thg or m^3 2 – thg/area or m^3/area 3 – evapo(thg)or evapo(m^3) 4 – evapo(time)
fertDosageMode	enum	Default fertilization mode **(Dream only)** 0 – None 1 – L/m^3 or g/thg 2 – sec/min 3 – mm:ss/m^3 or mm:ss/thg 4 – L/min or g/min 5 – L/prop or g/prop 6 – time 7 – L bulk or g/bulk
stopTimeAsMaxDuration	boolean	Use program stop time as program duration
haltOnRepeatedProblem	boolean	Halt programs when repeated flow detected **(Dream only)**
longSequences	boolean	Allow long sequences **(Dream only)**
dosageMultiplier	number	Common dosage coefficient (0-255)
sequentialFert	boolean	Enable sequential fertilization
localFertVV	boolean	Enable local fertilization mode litter/m3 **(Dream only)**
centralFertVV	boolean	Enable central fertilization mode litter/m3 **(Dream only)**
localFertTT	boolean	Enable local fertilization mode sec/min **(Dream only)**
centralFertTT	boolean	Enable central fertilization mode sec/min **(Dream only)**

localFertTV	boolean	Enable local fertilization mode m:s/m3 **(Dream only)**
centralFertTV	boolean	Enable central fertilization mode m:s/m3 **(Dream only)**
localFertVT	boolean	Enable local fertilization mode litter/min **(Dream only)**
centralFertVT	boolean	Enable central fertilization mode litter/min **(Dream only)**
localFertProp	boolean	Enable local proportional fertilization mode **(Dream only)**
centralFertProp	boolean	Enable central proportional fertilization mode **(Dream only)**
localFertTBulk	boolean	Enable local bulk fertilization mode by time **(Dream only)**
centralFertTBulk	boolean	Enable central bulk fertilization mode by time **(Dream only)**
localFertVBulk	boolean	Enable local bulk fertilization mode by volume **(Dream only)**
centralFertVBulk	boolean	Enable central bulk fertilization mode by volume **(Dream only)**
resetLeakageDaily	boolean	Extend leakage counter accumulation period to 24 hours
pulseBeforeFert	boolean	Enable a water pulse before fertilizer is expected
noFertDelay	number	Set custom delay for no fert. Pulse problem (0 – 9999 seconds)
sequentialMode	enum	Gradual valve opening delay type: 0 – by system 1 – by water source 2 – by irrigation line
sensorsLogHysteresis	number	Specifies how large (%) should be the change in order to be collected. Can be specified in fractions. Zero value forces collection of ALL data without tracing the changes.
flushFromLast	boolean	Enable start from the last flushed filter by the last run

timeAccEnabled	boolean	Enable collection of the time accumulations
areaResolutuionExtended	boolean	Enable extended area resolution (y.xxx)
diffMainValveDelay	boolean	Enable different open/close main valve delay
userDefinedId	boolean	User defined ID’s
isAreaExtendedReady	boolean	Indicates if the controller version is ready for the activation of the extended area resolution via Talgil applications.
supportsNetIDs	boolean	Indicates possibility of the controller radio module support the NetID definition

*Blue marked fields are modifiable.

6.4 Battery

URI	GET /targets/{id}/battery	
Description	Gets battery object	
Response	battery: {	
	online: boolean	*indicates that data is live or cached*
	state: enum	*0 – OK* *1 – Low* *2 – Dead*
	voltage: number	*current battery voltage*
	}	

6.5 Alarms

URI	GET /targets/{id}/alarms GET /targets/{id}/alarms/{id}	
Description	Gets all or specific alarm object	
Response	alarm: {	
	uid: string	*the object id*
	type: enum	*type of alarm, see appendix for the details*
	time: number	*the time when alarm was raised by controller, this is a UTC epoch time in ms*
	origin: ref	*reference to the object where alarm is originated from*
	}	

6.6 Main valves

URI	GET /targets/{id}/mainvalves GET /targets/{id}/mainvalves/{id}	
Description	Gets a collection or a specific main valve object	
Response	mainValve: {	
	uid: string	*unique object id*
	name: string	*user defined name*
	state: enum	*0 – closed* *1 – opened* *2 – communication error* *3 – opened manually* *4- not connected*
	online: boolean	*indicates that data is live or cached*
	card: number	*index of interface card*
	RTU: number	*index of RTU in the card*
	plugin: number	*Index of the plugin card on the RTU*
	output: number	*output number used*
	openMode: enum	*0 – no delay* *1 – before* *2 – after*
	closeMode: enum	*0 – no delay* *1 – before* *2 – after*
	openDelay: number	*Opening delay value in seconds*
	closeDelay: number	*Closing delay value in seconds*
	}	

6.7 Water sources

URI	GET /targets/{id}/sources GET /targets/{id}/sources/{id}	
Description	Gets a collection or a specific water source object	
Response	waterSource: {	
	uid: string	*unique object id*
	name: string	*water source name*
	state: enum	*0 – closed* *1 – working*
	online: boolean	*indicates that data is live or cached*
	flow: number	*Current water flow*
	waterMeter: object	*water meter if present*
	consumers: array	*collection of references to valves currently using the source*
	waterPumps: array	*collection of water pumps if present*
	}	

6.8 Water pumps

URI	GET /targets/{id}/pumps GET /targets/{id}/sources/{id}/pumps/{id}	
Description	Gets a collection or a specific water pump object	
Response	pump: {	
	uid: string	*unique object id*
	name: string	*user specified name or alias*
	state: enum	*0 – closed* *1 – opened* *2 – communication error* *3 – opened manually* *4 – not connected*
	online: boolean	*indicates that data is live or cached*
	card: number	*index of interface card*
	RTU: number	*index of RTU in the card*
	plugin: number	*Index of the plugin card on the RTU*
	output: number	*output number used*
	overloadState: enum	*0 – opened* *1 – closed* *2 – communication error* *3 – not connected*
	alarmState: enum	*0 – opened* *1 – closed* *2 – communication error* *3 – not connected*
	onoffState: enum	*0 – opened* *1 – closed* *2 – communication error* *3 – not connected*
	flow: number	*current flow value*
	}	

6.9 Water meters

URI	GET /targets/{id}/watermeters GET /targets/{id}/watermeters/{id} GET /targets/{id}/sources/{id}/watermeter GET /targets/{id}/lines/{id}/watermeter	
Description	Returns all, free, source or line water meter/s.	
Response	waterMeter: {	
	uid: string	*unique object id*
	name: string	*user specified name or alias*
	state: enum	*0 - no flow* *1 - flow exists* *2 - communication error* *3 - not connected* *4 - burst*
	online: boolean	*indicates that data is live or cached*
	card: number	*index of interface card*
	RTU: number	*index of RTU in the card*
	plugin: number	*Index of the plugin card on the RTU*
	input: number	*input number used*
	type: number	*type of water meter* *8 - drainage* *32 - free* *64 - line* *128 - source*
	ratio: number	*water meter ratio in corresponding units (m3/pulse or thg/pulse)*
	flow: number	*current flow value*
	maxFlow: number	*maximum measurable value*
	sensor: number	*index of analog sensor if connected*
	acc: number	*current water accumulation value*
	}	

6.10 Virtual water meters

URI	GET /targets/{id}/vwms GET /targets/{id}/vwms/{id}	
Description	Returns individual or all virtual water meters.	
Response	vwms: {	
	uid: string	*unique object id*
	name: string	*user specified name or alias*
	state: enum	*0 - no flow* *1 - flow exists* *2 - communication error* *3 - not connected* *4 - burst*
	online: boolean	*indicates that data is live or cached*
	flow: number	*current water flow*
	ratio: number	*virtual water meter ratio value*
	acc: number	*current water accumulation value*
	}	

6.11 Analog sensors

URI	GET /targets/{id}/sensors GET /targets/{id}/sensors/{id}	
Description	Returns all or individual analog sensors	
Response	sensor: {	
	uid: string	*unique object id*
	name: string	*user specified name or alias*
	state: enum	*0 - normal* *1 - measurement error* *2 - communication error* *3 - not connected*
	online: boolean	*indicates that data is live or cached*
	card: number	*index of interface card*
	rtu: number	*index of RTU in the card*
	plugin: number	*index of the plugin card on the RTU*
	mbu: number	*index of the MBU in the system*
	input: number	*input number used*
	type: enum	*type of sensor (see sensors types table)*
	units: enum	*measurement units (see units types table)*
	base: number	*1 - current* *2 - voltage* *3 - external* *4 - virtual* *5 - SDI* *6 - states/selector* *7 - ASCII string*
	minLimit: number	*lower limit*
	maxLimit: number	*higher limit*
	lowThreshold: number	*low threshold of the value*
	highThreshold: number	*high threshold of the value*
	readingDelay: number	*reading delay*
	readingRate: number	*how often the value will be read*
	dataSource: number	*data source identification, used with virtual sensors only*

	value: number	*current measurement value*
	updateTime: number	*time when the value of this sensor was updated, UTC, milliseconds*
	calculationType: enum	*type of the calculation performed by the sensor in case it's a virtual sensor:* *0 - string* *1 - daily accumulation* *2 - dew point* *3 - differential value* *4 - average value* *5 - satellite value* 6 - oversampling average 7 - evaporation 8 - chill accumulation 9 - Rainfall (analog increment)
	unitsDescriptor: string	*measurement unit descriptor*
	davis: boolean	*indicates whether the sensor is part of Davis weather station*
	baseSensor: array	*array of sensor indexes for virtual sensors*
	}	

6.12 Fertilization sites

URI	GET /targets/{id}/fertsites GET /targets/{id}/fertsites/{id} GET /targets/{id}/lines/{id}/fertsite	
Description	Returns fertilization site/s - all, central, local	
Response	fertSite: {	
	uid: string	*unique object id*
	name: string	*user specified name or alias*
	state: enum	*0 - not active* *1 - water before* *2 - fertigating*
	online: boolean	*indicates that data is live or cached*
	noFertOption: enum	*0 - stop current valve* *1 - stop fertilization* *2 - stop irrigation* *3 - inform only* *4 - skip to next valve*
	central: boolean	*indicates local or central fert site*
	position: number	*position of this site in the global index*
	booster: object	*booster object if present*
	ecph: object	*ECPH controller if present*
	ferts: array	*collection of fertilizer injectors*
	sets: [{ array	*fert sets library*
	name string	*user specified name of the fert set*
	mode[] array	*each element corresponds to the mode of respective fertilizer in the set*
	dosage[] array	*each element corresponds to the dosage of respective fertilizer in the set*
	}]	
	}	

6.12.1 Fertilizers

URI	GET /targets/{id}/ferts GET /targets/{id}/fertsites/{id}/fert/{id} GET /targets/{id}/lines/{id}/fertsite/fert/{id}	
Description	Returns fert injectors - all, central, local, individual or a collection	
Response	ferts: {	
	uid: string	*unique object id*
	name: string	*user specified name or alias*
	state: enum	*0 - closed* *1 - active* *2 - communication error* *3 - opened manually* *4 - not connected* *5 - no pulses* *6 - leakage* *7 - water before*
	online: boolean	*indicates that data is live or cached*
	card: number	*index of interface card*
	rtu: number	*index of RTU in the card*
	plugin: number	*index of the plugin card on the RTU*
	output: number	*output number used*
	site: string	*fertilization site uid*
	central: boolean	*indicates local or central site*
	flow: number	*current flow*
	maxFlow: number	*maximum measurable flow*
	acc: number	*current accumulated fertilizer volume*
	fertMeter: object	*fert meter if connected*
	agitator: object	*agitator if connected*
	shortestPulse: number	*Shortest period to operate the valve*
	ratio: number	*fert meter ratio value*
	}	

6.12.2 Fertmeters

URI	GET /targets/{id}/fertmeters GET /targets/{id}/fertsites/{id}/fertmeters/{id} GET /targets/{id}/lines/{id}/fertsite/fertmeters/{id}	
Description	Returns all, central or local fert meters, individual or collection	
	Fertmeter: {	
	uid: string	*unique object id*
	Name: string	*user specified name or alias*
	state: number	*the state of the fertmeter* *0 – no flow (flow=0)* *1 – flow exist (flow>0)* *2 - communication error* *3 - not connected*
	online: boolean	*indicates that data is live or cached*
	card: number	*index of interface card*
	plugin: number	*index of the plugin card on the RTU*
	rtu: number	*index of RTU in the card*
	input: number	*input number used*
	flow: number	*current flow*
	sensor: number	*sensor index number*
	}	

6.12.3 Agitators

URI	GET /targets/{id}/agitators GET /targets/{id}/fertsites/{id}/agitators/{id} GET /targets/{id}/lines/{id}/fertsite/agitators/{id}	
Description	Returns all, central or local fert meters, individual or collection	
Response	agitators: {	
	uid: string	*unique object id*
	name: string	*user specified name or alias*
	state: enum	*0 - closed* *1 - opened* *2 - communication error* *3 - opened manually* *4 - not connected* *6 - shorted*
	online: boolean	*indicates that data is live or cached*
	card: number	*index of interface card*
	rtu: number	*index of RTU in the card*
	plugin: number	*index of the plugin card on the RTU*
	output: number	*output number used*
	}	

6.12.4 Boosters

URI	GET /targets/{id}/boosters GET /targets/{id}/fertsites/{id}/booster GET /targets/{id}/lines/{id}/fertsite/booster	
Description	Returns boosters - all, central or local fert site.	
Response	booster: {	
	uid: string	*unique object id*
	name: string	*user specified name or alias*
	state: enum	*0 - closed* *1 - opened* *2 - communication error* *3 - opened manually* *4 - not connected*
	online: boolean	*indicates that data is live or cached*
	card: number	*index of interface card*
	rtu: number	*index of RTU in the card*
	plugin: number	*index of the plugin card on the RTU*
	output: number	*output number used*
	}	

6.12.5 ECPH

URI	GET /targets/{id}/ecph GET /targets/{id}/lines/{id}/fertsite/ecph	
Description	Depending on URI, the call returns ECPH controller of central or local fert site	
Response	ecph: {	
		the data depends from the ECPH interface settings.
	}	

6.13 Filtration sites

URI	GET /targets/{id}/filtersites GET /targets/{id}/filtersites/{id} GET /targets/{id}/lines/{id}/filtersite	
Description	Returns all, central or local filtration sites	
Response	filterSite: {	
	uid: string	*unique object id*
	name: string	*user specified name or alias*
	state: enum	*0 – closed* *1 – flushing* *2 – irrigating* *3 – looping*
	dsValve: object	*down stream valve if present*
	online: boolean	*indicates that data is live or cached*
	mode: enum	*mode of irrigation while flushing is active* *0 – Continues* *1 – Stop* *2 – Continues w/o fertilizing*
	dpDelay: number	*DP delay in seconds*
	maxLoops: number	*maximum number of flushing loops*
	intervalPlanned: number	*period between flushing cycles in seconds*
	intervalLeft: number	*time left before new cycle starts*
	flushTime: number	*flushing time in seconds*
	flushLeft: number	*flushing time left in seconds*
	pauseTime: number	*period between flushing valves in seconds*
	pauseLeft: number	*pause left in seconds*
	predwellTime: number	*delay between downstream and filter valve action in seconds*
	predwellLeft: number	*left delay between downstream and filter valve action in seconds*
	firstFilterTime: number	*special flushing time of first filter in seconds (0-363600)*
	betweenVolume: number	*water volume between flushing in m3 (thg)*
	accTime: number	*accumulated flushing started due time*

	accDP: number	*accumulated flushing started due DP*
	dpLoopsCount: number	*counter of consecutive starts due DP*
	currentFilter: number	*currently active flushing filter, 0 if none active*
	dpSensor: object	*differential pressure sensor*
	filters: array	*collection of all filters in the site*
	}	

6.13.1 Filters

URI	GET /targets/{id}/filtersites/{id}/filter/{id} GET /targets/{id}/lines/{id}/filtersite/filter/{id}	
Description	Returns central or local filters, individual or a collection	
Response	filter: {	
	uid: string	*unique object id*
	name: string	*user specified name or alias*
	state: enum	*0 – closed* *1 – opened* *2 – communication error* *3 – opened manually* *4 – not connected*
	online: boolean	*indicates that data is live or cached*
	card: number	*index of interface card*
	rtu: number	*index of RTU in the card*
	plugin: number	*index of the plugin card on the RTU*
	output: number	*output number used*
	}	

6.14 Irrigation lines

URI	GET /targets/{id}/lines/{id}	
Description	Returns collection or specific irrigation line object/s.	
Response	line: {	
	uid: string	*unique object id*
	name: string	*user given name to this line*
	state: enum	*0 – closed* *1 – irrigating* *2 – water after* *3 – fertigating* *4 – unknown* *5 – flushing* *6 – no pressure* *7 – frozen*
	online: boolean	*indicates that data is live or cached*
	lowFlowAction: enum	*how to deal with low flow indication* *0 – go to next work* *1 – ignore* *2 – wait if low flow*
	lowFlowResponse: number	*response time (sec) for low flow*
	highFlowAction: enum	*how to deal with high flow indication* *0 – go to next work* *1 – ignore* *2 – wait*
	highFlowResponse: number	*response time (sec) for high flow*
	waterBurstLimit: number	*water burst limit (0-99)*
	defaultSource: number	*default water source used with the line (0-4)*
	freezeReason: enum	*the reason line is frozen* *0 – not frozen* *1 – manually by command* *2 – low battery* *3 – network protection* *4 – AC power fail* *5 – repeated flow problem*
	mainValve: object	*index of the main valve attached to this line*

	centralFertSite: number	*index of central fert site connected to this line*
	centralFilterSite: number	*index of central filter site connected to this line*
	waterMeter: object	*water meter object of this line*
	pressureMeter: object	*pressure meter object of this line*
	filterSite: object	*local filter site of this line*
	fertSite: object	*local fert site of this line*
	valves: array	*collection of valves*
	}	

6.14.1 Valves

URI	GET /targets/{id}/valves GET /targets/{id}/lines/{id}/valves GET /targets/{id}/lines/{id}/valves/{id}	
Description	Returns all, line or a specific irrigation valve/s	
Response	valve: {	
	uid: string	*unique object id*
	name: string	*user given name*
	state: enum	*0 - closed* *1 - opened by program* *2 - communication error* *3 - opened manually* *4 - not connected* *5 - low pressure* *6 - wait program conflict* *7 - low flow* *8 - high flow* *9 - unresponsive on* *10 - unresponsive off* *11 - disabled by user* *12 - shorted output*
	online: boolean	*indicates that data is live or cached*
	card: number	*index of interface card*
	rtu: number	*index of RTU in the card*
	plugin: number	*index of the plugin card on the RTU*
	output: number	*io number used*
	line: number	*line number where valve belongs to*
	position: number	*position in line*
	minFlow: number	*Minimum flow, m3 (thg)*
	maxFlow: number	*Maximum flow, m3 (thg)*
	nomFlow: number	*Nominal flow, m3 (thg)*
	fillDelay: number	*Time during which the line is getting filled with water. During this period, flow and pressure violations are ignored. In minutes*
	dosageMode: enum	*Default dosage mode* *0 - time (h:m:s)* *1 - volume(m3)* *2 - volume/area* *3 - evaporation(volume)*

		4 – evaporation(time)
	area: number	*the area served by this valve*
	cropFactor: number	*crop factor (coefficient) for dosage correction*
	acc: number	*water accumulation by volume*
	accTime: number	*water accumulation by time*
	lastAcc: number	*last water accumulation by volume*
	lastAccTime: number	*last water accumulation by time*
	centralFertAcc: array	*central fertilizer accumulation by volume, per injector* **(Dream only)**
	localFertAcc: array	*local fertilizer accumulation by volume, per injector*
	lastCentralFertAcc: array	*last central fertilizer accumulation by volume, per injector*
	lastLocalFertAcc: array	*last local fertilizer accumulation by volume, per injector*
	fertLimitsLocal: array	*Maximum amount of fertilizer to be supplied by valve over season, liters (gallons). Each element in array corresponds to fert injector* **(Dream only)**
	fertLimitsCentral: array	*Maximum amount of fertilizer to be supplied by valve over season, liters (gallons). Each element in array corresponds to fert injector* **(Dream only)**
	specialty: enum	*enable/disable free valve 0 – normal function 1 – free valve function* ***(only for Sapir 2 controllers)***
	controlContact: number	*Reference to controller contact object*
	}	

*Blue marked fields are modifiable.

6.14.2 Pressure meters

URI	GET /targets/{id}/lines/{id}/pm	
Description	Gets a pressure meter of specified irrigation line	
Response	waterMeter: {	
	uid: string	*unique object id*
	name: string	*user specified name or alias*
	state: number	*0 - opened* *1 - closed*
	online: boolean	*indicates that data is live or cached*
	card: number	*index of interface card*
	rtu: number	*index of RTU in the card*
	plugin: number	*index of the plugin card on the RTU*
	input: number	*input number used*
	alarmLevel: number	*Pressure alarm level, 0-10 bar (0-150 psi)*
	sensor: ref	*Reference to analog sensor (optional)*
	}	

6.15 Programs

URI	GET /targets/{id}/programs/{id}	
Description	Returns specific or entire collection of irrigation programs	
Response	programs: {	
	uid: string	*unique object id*
	name: string	*user defined name of the program*
	state: enum	*1 - not ready* *2 - running+failure* *3 - wait pressure* *4 - wait* *5 - finished+failure* *6 - running* *7 - between cycles* *8 - frozen* *9 - wait condition* *10 - finished + left* *11 - scheduled-today* *12 - scheduled* *13 - ready + condition* *14 - finished* *15 - incomplete* *33 - cycling + error* *34 - running before reset* *35 - waiting before reset*
	online: bool	*indicates that data is live or cached*
	waitReason: enum	*0 - not defined* *1 - line has working program* *2 - valve(s) already activated* *3 - line frozen* *4 - line halted (flow problem)* *5 - water source power limit* *6 - flushing active* *7 - low pressure* *8 - program frozen* *9 - fert/fert conflict(local)* *10 - water - fert conflict(local)* *11 - fert/fert conflict(central)* *12 - water/fert conflict(central)*
	priority: number	*Program priority (if priority function is activated)*

	completion:	bool	*Allow completion of leftovers*
	startTime[]:	number	*Program start time in minutes since midnight, 0-1440. Program can have up to 6 start times.* **(Array is only by Sapir. Dream has only one start time by each program)**
	stopTime:	number	*Program stop time in minutes since midnight, 0-1440*
	lastStopTime:	number	*Time of the last stop (UTC ms), zero if last stop time not exist*
	lastRunTime:	number	*Time of the last run (UTC ms), zero if never ran*
	lastRunState:	enum	*State of the most recent run, see 'state' enumerator values*
	nextRunTime:	number	*Time of the next run (UTC ms), zero if not scheduled*
	cyclesPerStart:	number	*Number of irrigation cycles per one start, 0-9999*
	timeBetweenCycles:	number	*Time in seconds between irrigation cycles, 0-360000*
	daysCycle:	number	*Period between irrigation starts in days, 0-99*
	leftIrrigDay:	number	*Left days until the start of next irrigation cycle, 0-99*
	runList[]:	array	*Each element in array is irrigation day, each value indicates what to do on this day.* *0 - do nothing,* *1 - water only,* *2 - single cycle,* *3 - fertigation*
	currentValve:	number	*which of the valves in sequence currently irrigates if any, zero based index*
	currentDayCycle:	number	*Currently executed cycle (in cycle of days)*
	cyclesLeft:	number	*The left number of irrigations cycles*
	timeSinceCycleStart:	number	*The time since start of the current cycle*
	timeLeftBetweenCycles:	number	*Left time in seconds between irrigation cycles, 0-360000*
	startCondition:	enum	*start condition index for this program*

stopCondition:	enum	*stop condition index for this program*
enableCondition:	enum	*enable condition index for this program*
disableCondition:	enum	*disable condition index for this program*
sequence:	string	*Program sequence string in the following format: 1.1 & 1.2 > 1.3 + 1.4 ...*
valves:[{	**array**	***Collection of sequence valves***
valve:	ref	*Reference to actual irrigation valve*
state:	enum	*Current state of sequence valve* *0 - closed* *1 - frozen* *2 - wait* *3 - irrigating*
		4 - main delay *5 - water before* *6 - fertigating* *7 - water after* *8 - no fertilizer pulses* *9 - low flow* *10 - fertilizer leakage* *11 - high flow* *12 - fert+failure* *13 - wait flush* *14 - wait pressure* *15 - unconnected* *16 - ecph alarm*
flow:	number	*Current water flow*
waterDosageMode:	enum	*Water dosage mode* *0 - by time* *1 - by volume* *2 - by area* *3 -evaporation* *4 - evaporation + time*
waterPlanned:	number	*Planned water dosage according to the mode the value can be time in seconds (integer) or volume in m3 (gallons) as float.*
waterCalc:	number	*Water dosage calculated in the format of planned*
waterLeft:	number	*Water dosage left in the format of planned*
waterBeforeMode:	enum	*Water before mode* **(Dream only)** *0 - by time* *1 - by volume*

waterBeforeLocal:	number	*Water before local fertilization*
waterBeforeCentral:	number	*water before central of valve* **(Dream only)**
waterBeforeSpecial:	number	*water before special of valve before first fert. injector* **(Dream only)**
waterBeforeAdditional:	number	*water before additional of valve before second fert. injector* **(Dream only)**
waterAfter:	number	*Water after fertilization, in the format of planned*
localFertMode[]:	array	*Mode of local fertilization, array item corresponds the local injector and its value to the actual mode enumerator.*
localFertPlanned[]:	array	*Planned local fertilization, array item corresponds the local injector and its value to the actual dosage*
localFertLeft[]:	array	*Left local fertilization, array item corresponds the local injector and its value to the actual left.*
centralFertMode[]:	array	*the mode of central fertilization, array item is enum which correspond to the fertigation mode of each injector* **(Dream only)**
centralFertPlanned[]:	array	*the planned fertilization for central, array item corresponds to central injector planned fertilization* **(Dream only)**
centralFertLeft[]:	array	*the left fertilization for central, array item corresponds to central injector planned fertilization* **(Dream only)**
localFertScaled:	array	*scale for local fertilization, array item corresponds to scale of local fertilizer dosage (0-EC, 1-pH)*
centralFertScaled:	array	*scale for central fertilization, array item corresponds to scale of central fertilizer dosage (0-EC, 1-pH)*
ecLocalPlanned:	number	*the local fertilization ec planned*
phLocalPlanned:	number	*the local fertilization ph planned*
ecCentralPlanned	number	*the central fertilization ec planned* **(Dream only)**

	phCentralPlanned	number	*the central fertilization ph planned* **(Dream only)**
	ecLocalLast:	number	*the local fertilization ec last*
	phLocalLast:	number	*the local fertilization ph last*
	ecCentralLast:	number	*the central fertilization ec last*
	phCentralLast:	number	*the central fertilization ph last*
	ecLocalAverage:	number	*the local fertilization ec average*
	phLocalAverage:	number	*the local fertilization ph average*
	ecCentralAverage:	number	*the central fertilization ec average*
	phCentralAverage:	number	*the central fertilization ph average*
	fsetLocal:	number	*The index of the fertilization set used for local fert site*
	fsetCentral:	number	*The index of the fertilization set used for current central fert site*
	}]		
	}		

*Blue marked fields are modifiable.

6.16 Interface cards

URI	GET /targets/{id}/interfaces/{id}		
Description	Gets individual or all interface card objects		
Response	interface: {		
	uid:	string	*unique object id*
	name:	string	*user specified name or alias*
	state:	enum	*0 – ok* *1 – communication error* *2 – RTU error* *3 – low battery* *4 – RF test in progress*
	online:	boolean	*indicates that data is live or cached*
	mode:	enum	*0 – normal work* *1 – counting activated*
	type:	enum	*type of card* *0 – None* *1 – DC* *2 – 2 wire* *3 – 4 wire* *4 – RF* *6 – AC* *7 – Analog* *8 – EC/pH* *11 – RF G5 (RF generation 5)* *12 – 2W G2 (2 – wire generation 2)* *13 – Cellular* *14 – DC Fast* *15 – AC Fast* *16 – Modbus* *17 – Analog Fast* *18 – pH/EC Fast*
	address:	number	*card address*
	param1:	number	*configuration parameter –* *in case of AC/DC interface:* *Out/In configuration:* *0 – 16 / 8* *1 – 32 / 16* *2 – 6 / 4* *in case of RF interface:* *polling rate:* *0 – 10 sec* *1 – 5 sec* *2 – 2.5 sec* *3 – 1.2 sec* *4 – RF test mode*

		in case of pH/EC interface: *index of associated fertigation site* *in case of Analog interface:* *polling rate:* *0 – usual* *1 – Davis weather station* *2 – THD unit* *3 – external (proxy) interface* *in case of Analog interface:* *0 – usual* *1 – Davis weather station* *2 – THD unit* *3 – external (proxy) interface* *in case of Analog fast interface* *0 – usual* *1 – Davis weather station* *2 – SDI* *3 – EC/pH monitor* *Modbus interfaces:* *0 – not ready* *1 – model loaded*
	param2: number	*additional parameter – relevant for NMC (Netafim RTU interface)*
	firmware: number	*firmware version*
	errorCounters: string	*string of error counters*
	RTUs: array	*collection of RTUs if available*
	netID: number	*number of the network ID of the RFG5 master*
	}	

6.16.1 RTU

URI	GET /targets/{id}/rtus/ GET /targets/{id}/interfaces/{id}/rtus/{rtu}	
Description	Gets individual RTU object from specified interface or batch of all RTU of the controller at once. I case of RTUs we have tree structure and number of the RTU in the request will be taken not as second part of the "uid" but from the index of the RTUs in the interface shown in the parameter "rtu". Please mention it in your requests.	
Response	RTU: {	
	uid: string	*unique object id*
	id: number	*position of RTU in the interface card map*
	name: string	*user specified name or alias*
	state: enum	*0 - ok* *1 - interface error* *2 - communication error* *3 - RTU low battery* *4 - RTU under test* *5 - not connected/disabled*
	online: boolean	*indicates that data is live or cached*
	type: number	*reserved*
	card: number	*address of interface card*
	rtu: number	*index of RTU in the card*
	plugins: array	*collection of plugins if available*
	}	

6.16.2 Plugin

URI	GET /targets/{id}/plugins/ GET /targets/{id}/interfaces/{id}/rtus/{rtu}/plugins/{plugin}	
Description	Gets individual plugin object from specified interface and RTU or batch of all plugins of the controller at once. By the plugin number works similar logic as by RTUs because of the same tree structure. Number of the plugin on the specific RTU is shown in the parameter "plugin". Please mention it by your requests.	
Response	plugin: {	
	uid: string	*unique object id*
	name: string	*user specified name or alias*
	state: enum	*status of the plug card:* *0 – ok* *1 – interface communication error* *2 – RTU communication error* *5 – RTU disabled* *6 – plug communication error* *7 – plug disabled*
	online: boolean	*indicates that data is live or cached*
	type: enum	*plug card type:* *0 – not defined* *1 – 4 ANA* *2 – SDI* *3 – ECPH* *4 – MODBUS* *5 – I/O 8/4* *6 – Davis weather station* *7 – pH-EC control unit*
	card: number	*address of interface card*
	rtu: number	*index of RTU in the card*
	plugin: number	*plugin card index on the RTU*
	}	

6.17 Conditions

URI	GET /targets/{id}/conditions/{id}	
Description	Gets individual or all conditions	
Response	condition: {	
	uid: string	*unique object id*
	name: string	*user given name or alias*
	state: enum	*0 - false* *1 - true*
	online: boolean	*1 indicates that data is live or cached*
	enabled: boolean	*indicates whether the condition is used*
	type: enum	*type of condition. This field is obligatory and must be present in each condition modification request for each modifiable condition.*
	value: number	*analog value if applicable*
	expression: string	*expression of the combined conditions* **(Dream only)**
	target: string	*reference to the condition object. (In case of modification use command "**target.index**" to define the element number. Container will be defined according to the condition type). This field is obligatory and must be present in each condition modification request for each modifiable condition.*
	duration: number	*time span when state transition is effective (seconds)*
	fromTime: number	*beginning of the local time when condition is effective. Minutes since midnight*
	untilTime: number	*end of the local time when condition is effective. Minutes since midnight*
	events: boolean	*indicates that condition can be used for triggering system events*
	}	

*Blue marked fields are modifiable.
****bold*** marked fields are obligatory parameters and have to be present in each condition modification request for modifiable condition.

6.18 Satellites

URI	GET /targets/{id}/satellites GET /targets/{id}/satellites/{id}	
Description	Gets all or individual satellites	
Response	satellite: {	
	uid: string	*unique object id*
	name: string	*user given name or alias*
	state: number	*0 - closed* *1 - opened* *2 - communication error* *3 - opened manually* *4 - not connected*
	online: boolean	*indicates that data is live or cached*
	card: number	*index of interface card*
	rtu: number	*index of RTU in the card*
	plugin: number	*index of the plugin card on the RTU*
	mbu: number	*index of the MBU in the system*
	output: number	*output number used*
	members: array	*references to output objects making up the satellite*
	}	

6.19 Contacts

URI	GET /targets/{id}/contacts/{id}	
Description	Gets individual or all contacts	
Response	contact: {	
	uid: string	*unique object id*
	card: number	*index of interface card*
	rtu: number	*index of RTU in the card*
	plugin: number	*index of the plugin card on the RTU*
	mbu: number	*index of the MBU in the system*
	input: number	*input number used*
	name: string	*user specified name or alias*
	state: number	*0 - opened* *1 - closed*
	type: enum	*Specifies the type of contact this is* *0=OVERLOAD* *1=ALARM* *2=ONOFF* *3=GENERAL_DRY* *4=BOTTOM_LEVEL_FLOAT* *5=MIDDLE_LEVEL_FLOAT* *6=TOP_LEVEL_FLOAT* *7=THERMOSTAT* *8=UNKNOWN*
	sensor: number	*index of the attached analog sensor*
	}	

6.20 Weather station

URI	GET /targets/{id}/weather	
Description	Gets weather station object	
Response	weather: {	
	uid: string	*unique object id*
	state: number	*0 - ok* *1 - communication error* *-1 - not connected*
	online: boolean	*indicates that data is live or cached*
	radiation: number	*value from sensor if present*
	uvRadiation: number	*value from sensor if present*
	windSpeed: number	*value from sensor if present*
	windDirection: number	*value from sensor if present*
	atmPressure: number	*value from sensor if present*
	humidity: number	*value from sensor if present*
	temperature: number	*value from sensor if present*
	dewPoint: number	*value from sensor if present*
	rainRate: number	*value from sensor if present*
	dailyRain: number	*value from sensor if present*
	evapotranspiration: number	*value from sensor if present*
	}	

6.21 Commands

All commands are sent to controller by means of POST requests with optional set of parameters and content as specified individually for each command below. The response to all commands is always JSON carrying a meaningful response code and message:

```
response:{
rc : number,
message: text }
```

6.21.1 Controller commands

URI	POST /targets/{id}	
Parameters	command	*freeze, release*
Examples	POST /targets/123456?command=freeze *will freeze controller 123456*	

6.21.2 Program commands

URI	POST /targets/{id}/programs/{id}	
Parameters	command	*start, stop, skip, freeze, release, releaseAndTerminate, remove, duplicate*
	from	*used with **'start'** command and specifies the index of sequence unit to start from, if not specified, the **first unit is assumed as 1.***
	mode	*used with **'start'** command and specifies start mode as follows: 0-normal,1-single cycle,2-with lefts,3-without ferts*
Examples	Assuming the program sequence 1.1 & 1.2 > 1.3 + 1.4 > 1.5 POST /targets/123456/programs/23?command=start&from=2&mode=3 *will start program 23 from "1.3 + 1.4" without fertilizer* POST /targets/123456/programs/5?command=start&mode=1 *will start a single cycle of program 5 from "1.1&1.2"* POST /targets/123456/programs/6?command=freeze *will freeze program 6* POST /targets/123456/programs/6?command=remove *will remove program 6 from controller* POST /targets/123456/programs/6?command=duplicate *will duplicate program 6 by creating new identical program*	

6.21.3 Output commands

The following command can be issued on any type of output support by controller. Note that outputs are referenced by flat (not relative!) index, e.g. valve 25 is not the valve in particular line but simply a 25th valve in controller image. The following types of outputs can be used in the URL: valves, mainvalves, pumps, ferts, filters, boosters, agitators.

URI	POST /targets/{id}/{outputType}/{id}	
Parameters	command	*open, close*
Examples	POST /targets/123456/valves/25?command=open *will open valve 25, note the flat index of the valve (not related to the irrigation line)* POST /targets/123456/filters/21?command=close *will close filter 21, note the flat index of the filter (not related to the filter site)*	

6.21.4 Clearing alarms

URI	POST /targets/{id}/alarms/{id}	
Parameters	command	reset
Examples	POST /targets/123456/alarms/1?command=reset *will attempt to clear alarm 1*	

6.22 Modifications

This section describes how to modify controller data. There are references to specific controller IDD for the details on which indexes to use and what values are acceptable.

6.22.1 Creating new programs

Use the following call to create irrigation programs.

URI	POST /targets/{id}/programs/new	
Parameters	sequence	*string representation of program sequence. The format is as follows: 1.1A > 1.2B + 1.3 > 1.4 & 1.5. Note that water source letter is optional.*
	name	*name for the new program*
Examples	POST /targets/1/programs/new?sequence=1.1A>1.2B>1.2+1.4&name=foo	

6.22.2 Modifying programs

This call can be used to modify program parameters defined in controller IDD.

URI	POST /targets/{id}/programs/{id}/modify	
Parameters	index	*the index of the field to change, see Dream IDD programs section*
	value	*the value to apply to this field*
Examples	POST /targets/1/programs/2/modify?index=105&value=480 *Set program 2 start time in controller 1 to 8:00AM*	
	POST /targets/1/programs/2/modify?index=107&value=10 *Set program 2 number of cycles in controller 1 to 10*	
	POST /targets/1/programs/2/modify?index=221&value=1.1>1.2>1.3 *Modifies sequence of program 2 in controller 1 to 1.1>1.2>1.3*	

6.22.3 Modifying program dosage

This call can be used to modify program sequence parameters defined in controller IDD.

URI	POST /targets/{id}/programs/{id}/sequence/{vid}/modify	
Parameters	{vid}	*This is the sequence valve number. For example, when sequence is **1.1 & 1.2 > 2.1 + 2.2**, to refer to valve **2.1**, the **vid = 3** (the position of the valve in sequence)*
	index	*the index of the field to change, see Dream IDD sequence element section*
	fert	*when modifying fertilizer related index then this parameter is the fertilizer injector number*
	value	*the value to apply to this field*
Examples	POST /targets/1/programs/2/sequence/3/modify?index=105&value=1 *This will target controller #1 program #2.* *Provided that sequence of this program is **1.1 & 1.2 > 2.1 + 2.2**, it will modify **water dosage mode** of valve **2.1** to **VOLUME***	
	POST /targets/1/programs/3/sequence/2/modify?index=106&value=120.5 *This will target controller #1 program #3.* *Provided that sequence is **1.1 & 1.2 > 2.1 + 2.2**, this will modify **water dosage planned** of valve **1.2** to **120.5 m3** (if volumes are metric for this controller, otherwise thousands of gallons)*	
	POST /targets/1/programs/3/sequence/2/modify?index=113&fert=1&value=5 *This will target controller #1 program #3.* *Provided that sequence is **1.1 & 1.2 > 2.1 + 2.2**, it will modify **local fert dosage mode** of **fert injector #1** serving valve **1.2** to **"Proportional volume"***	

6.22.4 Modifying valve properties

This call can be used to modify valve parameters specified in controller IDD.

URI	POST /targets/{id}/lines/{id}/valves/{id}/modify	
Parameters	index	*index of the field to change, see Dream IDD specification*
	value	*the value to apply to this field, if index is an array, comma separated string is expected*
Examples	POST /targets/1/lines/1/valves/1/modify?index=105&value=50 *Set the valve crop factor to 50%*	
	POST /targets/1/lines/1/valves/1/modify?index=109&value=0,0,0,0,0,0 *Resets local fertilizer accumulations for all six injectors*	

6.22.5 Virtual sensors updates

It is possible to modify values of analog sensors which are specified in controller image as virtual PESSL sensors, e.g. sensors having an external data source reference starting with “PESSL.XXX...”. Note that sensor values are sent as part of body payload in JSON format, see example below.

Data source identifier consists of three parts - the data source **type**, the data source **id** and the sensor **moniker**. Parts are separated with the **dot character**. Server will ignore requests directed to non-existent sensors. The look-up for physical sensors in the controller is based on the data source identifiers starting with “PESSL.XXX...”.

Sensor values should come as floating-point numbers.

URI	`POST /targets/{id}/sensors/modify`
Body payload	`{` `  “type.id.moniker”: value,` `  “type.id.moniker”: value,` `...` `}`
Examples	`POST /targets/1/sensors` `{ "PESSL.00206445.10000_X_X_16385_avg" : 10.2,` `  "PESSL.00206445.ETo[mm]" : 122.2` `}`

6.22.6 Program modification

It is a new request which allows to use the parameter name as variables for this request and to change several variables with one request. Parameters which can be modified are marked in the appropriate chapter with the blue color. Please look here for details.
Everything what is inside of braces **{}** should be sent in the request body JSON format.
The parameters in the Body Payload will be processed in order from top to bottom. This means that the order of the parameter in the Body Payload can be important. Please be aware of it.

URI	POST /targets/{id}/programs/{id}/modifyFields	
Parameters	parameter	*the parameter of the field to change, it can be taken from the JSON of the GET request of the appropriate section or the whole image or from this document*
	value	*the value to apply to this field*
Body Payload	{ "parameter": value, "parameter": value, "parameter": "value1, value2, value3", "parameter": value }	
Example	POST /targets/1/programs/1/modifyFields { "startTime": 601, "sequence": "1.1A > 1.2A > 1.3 > 1.4A > 1.5A", "runList": "3, 3, 3, 3, 3, 3, 3", "stopCondition": 2 } *Set program 1 start time in the controller 1 to 10:01AM* *Set program 1 sequence to 1.1A > 1.2A > 1.3 > 1.4A > 1.5A* *Set program 1 run list to "F" for all 7 days* *Set condition number 2 as a stop condition for the program 1*	

6.22.7 Program dosage (Sequence element) modification

This new request allows the parameter to be used as variable as well and will be used to modify dosages for the selected sequence element/valve. It is possible to change several parameters with one request.
Parameters which can be modified are marked in the appropriate chapter with the blue color.
Please look here for details (inside of the **valves** array).
Everything what is inside of braces **{}** should be sent in the request body JSON format.
The parameters in the Body Payload will be processed in order from top to bottom. This means that the order of the parameter in the Body Payload can be important. Please be aware of it.

URI	POST /targets/{id}/programs/{id}/sequence/(vid)/modifyFields	
Parameters	{vid}	*This is the sequence valve number. For example, when sequence is **1.1 & 1.2 > 2.1 + 2.2**, to refer to valve **2.1**, the **vid = 3** (the position of the valve in sequence)*
	parameter	*the parameter of the field to change, it can be taken from the JSON of the GET request of the appropriate section or the whole image or from this document*
	value	*the value to apply to this field*
Body Payload	{ "parameter": value, "parameter": value, "parameter": value, "parameter[x]": value }	
Example	POST /targets/1/programs/1/sequence/1/modifyFields { "waterDosageMode": 0, "waterPlanned": 61, "waterBeforeLocal": 10, "localFertMode[1]" : 6, "localFertPlanned[1]" : 100, "localFertMode[2]" : 6, "localFertPlanned[2]" : 47 } *Set Water Dosage Mode to time for the 1st valve of the sequence element* *Set Water Planned to 61 seconds (according to the set dosage mode)* *Set Water Before Local to 10 seconds* *Set Local Fert Mode for the 1st fertilizer injector to Time Bulk* *Set Local Fert Planned for the 1st fertilizer injector to 100 seconds* *Set Local Fert Mode for the 2nd fertilizer injector to Time Bulk* *Set Local Fert Planned for the 2nd fertilizer injector to 47 seconds*	

6.22.8 Program dosages (Sequence elements) modification (batch)

This request is similar to the previous one but here it is possible to modify several sequence elements with one request. This request allows to use parameters as variables.
Parameters which can be modified are marked in the appropriate chapter with the blue color. Please look here for details (inside of the **valves** array).
Everything what is inside of braces **{}** should be sent in the request body JSON format.
The parameters in the Body Payload will be processed in order from top to bottom. This means that the order of the parameter in the Body Payload can be important. Please be aware of it.

URI	POST /targets/{id}/programs/{id}/sequence/modifyMany	
Parameters	{vid}	*This is the sequence valve number. For example, when sequence is **1.1 & 1.2 > 2.1 + 2.2**, to refer to valve **2.1**, the **vid = 3** (the position of the valve in sequence)*
	parameter	*the parameter of the field to change, it can be taken from the JSON of the GET request of the appropriate section or the whole image or from this document*
	value	*the value to apply to this field*
Body Payload	{ "1":{ "parameter": value, "parameter": value, "parameter": value, "parameter[x]": value } "x":{ "parameter": value, "parameter": value, "parameter": value, "parameter[x]": value } }	
Example	POST /targets/1/programs/1/sequence/modifyMany { "1":{ "waterDosageMode": 0, "waterPlanned": 61 }, "2":{ "waterDosageMode": 1, "waterPlanned": 12 }, "5":{ "waterDosageMode": 3	

	} } *Set Water Dosage Mode to time for the 1st element of the sequence* *Set Water Planned to 61 seconds (according to the set dosage mode)* *Set Water Dosage Mode to volume for the 2nd element of the sequence* *Set Water Planned to 12 m3 (according to the set dosage mode)* *Set Water Dosage Mode to evaporation for the 5th element of the sequence*

6.22.9 Condition modification

This request allows to modify the condition using parameters as variables.
Parameters which can be modified are marked in the appropriate chapter with the blue color.
Please look here for details.
Everything what is inside of braces **{}** should be sent in the request body JSON format.
The parameters in the Body Payload will be processed in order from top to bottom. This means that the order of the parameter in the Body Payload can be important. Please be aware of it.

URI	POST /targets/{id}/conditions/{id}/modifyFields	
Parameters	parameter	*the parameter of the field to change, it can be taken from the JSON of the GET request of the appropriate section or the whole image or from this document*
	value	*the value to apply to this field*
Body Payload	{ "type": value, "target.index": value, "parameter": value, "parameter": value }	
Example	POST /targets/1/conditions/1/modifyFields { "type": 6, "target.index": 3, "duration": 10, "events": 1 } *Set condition 1 type "Contact close". The defined contact should be closed then condition becomes "TRUE"* *Set condition 1 target contact number 3* *Set condition 1 duration 10 seconds. Contact should be close for 10 seconds then the condition becomes "TRUE"* *Enable condition 2 event creation by change of the condition status*	

- For the conditions types 1 – 18 obligatory parameters for each request are "type" and "target.index".
- For the conditions types 19 – 20 obligatory parameters for each request are "type" and "expression" (**DREAM Only**).
- When the condition is not enabled and will not be to change the status and basically it will not work. Please be aware of it.

6.22.10 Conditions modification (batch)

This request allows to modify several conditions at once using parameters as variables. Parameters which can be modified are marked in the appropriate chapter with the blue color. Please look here for details.
Everything what is inside of braces **{}** should be sent in the request body JSON format.
The parameters in the Body Payload will be processed in order from top to bottom. This means that the order of the parameter in the Body Payload can be important. Please be aware of it.

URI	POST /targets/{id}/conditions/modifyMany	
Parameters	parameter	*the parameter of the field to change, it can be taken from the JSON of the GET request of the appropriate section or the whole image or from this document*
	value	*the value to apply to this field*
Body Payload	{ "1":{ "type": value, "target.index": value, "parameter": value, "parameter": value } "x":{ "type": value, "target.index": value, "parameter": value, "parameter": value } }	
Example	POST /targets/1/conditions/modifyMany { "1":{ "type": 1, "target.index": 2, }, "2":{ "type": 17, "target.index": 20, "value": 50, "enabled": true }, "5":{ "type": 19, "expression": "(1+2)&(3+4)", "enabled": true } }	

	Set for the condition 1 type "Program is running" and Program 2 as a target. It means that condition becomes TRUE when Program 2 is running *Set for the condition 2 type "Sensor higher than" and Sensor 20 as a target. Value for the condition activation will be set as 50 and condition will be enabled. It means that this condition will be enabled and becomes TRUE when the value of the Sensor 20 will be over then 50* *Set for the condition 5 type "Combined condition TRUE" and Expression for this condition should be set as "(1+2)&(3+4)"and enable this condition. It means that condition becomes TRUE when Condition 1 or Condition 2 and at the same time Condition 3 or Condition 4 should be TRUE*

- For the conditions types 1 – 18 obligatory parameters for each request are "type" and "target.index".
- For the conditions types 19 – 20 obligatory parameters for each request are "type" and "expression" (**DREAM Only**).
- When the condition is not enabled and will not be to change the status and basically it will not work. Please be aware of it.

6.22.11 Modifying valve properties (new)

It is a new request which allows to use the parameter name as variables for this request and to change several variables with one request. Parameters which can be modified are marked in the appropriate chapter with the blue color. Please look here for details.
Everything what is inside of braces **{}** should be sent in the request body JSON format.
The parameters in the Body Payload will be processed in order from top to bottom. This means that the order of the parameter in the Body Payload can be important. Please be aware of it.

URI	`POST /targets/{id}/lines/{id}/valves/{id}/modifyFields`	
Parameters	parameter	*the parameter of the field to change, it can be taken from the JSON of the GET request of the appropriate section or the whole image or from this document*
	value	*the value to apply to this field*
Body Payload	`{` `  "parameter": value,` `  "parameter": value,` `  "parameter": value,` `  "parameter": value` `}`	
Example	`POST /targets/1/lines/1/valves/1/modifyFields` `{` `  "nomFlow": 100,` `  "dosageMode": 1,` `  "area": 10,` `  "cropFactor": 105` `}` *Set nominal flow of the valve to 100* *Set default dosage mode of the valve to volume(m3)* *Set area of the valve to 10* *Set crop factor index to 105*	

- In case if you will write parameters “minFlow” or/and “maxFlow” before parameter “nomFlow” you will have at the end automatically calculated values of “minFlow” or/and “maxFlow”. To do it properly both these parameters should come after the parameter “nomFlow”.

6.22.12 Modifying valve properties (batch)

It is a new request which allows to use the parameter name as variables for this request and to change several variables with one request. Parameters which can be modified are marked in the appropriate chapter with the blue color. Please look here for details.
Everything what is inside of braces **{}** should be sent in the request body JSON format.
The parameters in the Body Payload will be processed in order from top to bottom. This means that the order of the parameter in the Body Payload can be important. Please be aware of it.

URI	POST /targets/{id}/lines/{id}/valves/modifyMany	
Parameters	parameter	*the parameter of the field to change, it can be taken from the JSON of the GET request of the appropriate section or the whole image or from this document*
	value	*the value to apply to this field*
Body Payload	{ "1":{ "parameter": value, "parameter": value, "parameter": value, "parameter": value } "x":{ "parameter": value, "parameter": value, "parameter": value, "parameter": value } }	
Example	POST /targets/1/ lines/{id}/valves/modifyMany { "1":{ "cropFactor": 110, "fillDelay": 60 }, "2":{ "cropFactor": 110, "area": 12 }, "5":{ "cropFactor": 110, "nomFlow": 60 }, "21":{ "cropFactor": 110, "acc": 5660 } }	

	Set crop factor index to 110 for the valves 1, 2, 5 and 21 *Set Valve 1 fill up delay as 60 seconds* *Set Valve 2 area as 12* *Set Valve 5 nominal flow as 60 m3/h (min and max flow will be recalculated for this valve automatically)* *Set Valve 21 accumulations as 5660*

- In case if you will write parameters “minFlow” or/and “maxFlow” before parameter “nomFlow” you will have at the end automatically calculated values of “minFlow” or/and “maxFlow”. To do it properly both these parameters should come after the parameter “nomFlow”.

6.22.13 Modifying controller settings (Dealer Definitions)

It is a new request which allows to use the parameter name as variables for this request and to change several variables with one request. Parameters which can be modified are marked in the appropriate chapter with the blue color. Please look here for details.
Everything what is inside of braces **{}** should be sent in the request body JSON format.
The parameters in the Body Payload will be processed in order from top to bottom. This means that the order of the parameter in the Body Payload can be important. Please be aware of it.

URI	`POST /targets/{id}/dream/modifyFields`	
Parameters	parameter	*the parameter of the field to change, it can be taken from the JSON of the GET request of the appropriate section or the whole image or from this document*
	value	*the value to apply to this field*
Body Payload	`{` `  "parameter": value,` `  "parameter": value,` `  "parameter": value,` `  "parameter": value` `}`	
Example	`POST /targets/1/dream/modifyFields` `{` `  "groups": true,` `  "longSequence": false,` `  "conditions": true,` `  "evapoControl": true` `}` *Enable the usage "Named Groups"* *Disable the Long Sequence usage on the controller* *Enable usage of the conditions* *Enable Evaporation Control*	

6.23 Data analysis

SPOT server is backed with database storing data logs amongst other things. There are roughly two kinds of data available through this API - observations and consumptions. Observations are the actual measurements collected from controller per individual object like analog sensors, water meters, valves, etc. In other words - raw data. Consumptions on the other hand are calculated values based on accumulations. More details below.

6.23.1 Raw data query (batch)

This call is suitable for those who want observations for ALL objects of specific type. The caller will provide the time range and type of observation requested. Server will respond with a list of time-value pairs. The list is ordered by time, where time is the actual measurement time of requested signal.

URI	GET /targets/{id}/{type}/log {id} is the serial number of controller {type} is type of object (sensors, watermeters, valves, etc)	
Parameters	from	*epoch time (UTC, milliseconds)*
	until	*epoch time (UTC, milliseconds)*
	otype	*observation type (see details)*
Response JSON	[uid: [{	*array of time-value pairs mapped to the object uid*
	time: number	*epoch time (UTC, milliseconds)*
	value: number	*measured value*
	}],...]	
Examples	GET /targets/123/sensors/log? from=X&until=Y&otype=2	*gets **analog** data of all sensors in time range X-Y*
	GET /targets/123/watermeters/log? from=X&until=Y&otype=2	*gets **flow** data of all watermeters in time range X-Y*
	GET /targets/123/valves/log? from=X&until=Y&otype=8	*Gets **water time accumulation** of all valves in time range X-Y*
	GET /targets/123/ferts/log? from=X&until=Y&otype=5	*Gets **fert volume accumulation** of all valves in time range X-Y*

6.23.2 Raw data queries (individual objects)

This call is suitable for those who want observations for an individual object indexed within the type namespace. The caller will provide the time range and type of observation requested. Server will respond with a list of time-value pairs. The list is ordered by time, where time is the actual measurement time of requested signal.

URI	GET /targets/{id}/{type}/{oid}/log {id} is the serial number of controller {type} is type of object (sensors, watermeters, etc) {oid} is the orderly number of object		
Parameters	from	*epoch time (UTC, milliseconds)*	
	until	*epoch time (UTC, milliseconds)*	
	otype	*observation type (see details)*	
Response JSON	[{	*array of time-value pairs*	
	time: number	*epoch time (UTC, milliseconds)*	
	value: number	*measured value*	
	},...]		
Examples	GET /targets/123/sensors/4/log?from=X&until=Y&otype=ANALOG	*gets data of analog sensor #4 in time range X-Y*	
	GET /targets/123/watermeter/2/log?from=X&until=Y&otype=ANALOG	*gets flow data of free water meter #2 in time range X-Y*	
	GET /targets/123/conditions/3/log?from=X&until=Y&otype=DIGITAL	*gets data of condition #3 in time range X-Y*	
	GET /targets/123/contact/45/log?from=X&until=Y&otype=DIGITAL	*gets data of contact #45 in time range X-Y*	
	GET /targets/123/programs/2/log?from=X&until=Y&otype=DIGITAL	*gets program 2 starts/stops in time range X-Y*	
	GET /targets/123/watermeter/1/log?from=X&until=Y&otype=WATER_ACC	*Gets **water volume accumulation** of first free watermeter in time range X-Y*	

6.23.3 Raw data queries (line objects)

This variation is similar to the previous but allowing access to data of line objects such as valves, water meters, pressure meters, etc.

URI	GET /targets/{id}/lines/{lid}/{type}/log GET /targets/{id}/lines/{lid}/{type}/{pos}/log {id} is the serial number of controller {type} is type of object (valves, watermeters, pm's, main valves) {pos} is the orderly number of object in position	
Parameters	same as above	
Response JSON	same as above	
Examples	GET /targets/123/lines/4/**wm**/log? from=X&until=Y&otype=ANALOG	*gets data of watermeter in line 4, in time range X-Y*
	GET /targets/123/lines/4/**pm**/log? from=X&until=Y&otype=ANALOG	*gets data of watermeter in line 4, in time range X-Y*
	GET /targets/123/lines/5/**mainvalve**/log? from=X&until=Y&otype=ANALOG	*gets data of main valve attached to line 5, in time range X-Y*
	GET /targets/123/lines/4/**valves**/2/log? from=X&until=Y&otype=DIGITAL	*gets valve 4.2 data in time range X-Y*
	GET /targets/123/lines/1/**valves**/2/log? from=X&until=Y&otype=WATER_TIME_ACC	*Gets second valve in first line water time accumulations in time range X-Y*

6.23.4 Raw data queries (fertilization)

These calls for raw data are specifically mapped for objects in central and local fertilization systems, such as sites, injectors, boosters, agitators, etc.

URI	**Central fertilization calls:** GET /targets/{id}/fertsites/{sid}/{type}/log GET /targets/{id}/fertsites/{sid}/{type}/{pos}/log **Local fertilization calls:** GET /targets/{id}/lines/{lid}/fertsite/log GET /targets/{id}/lines/{lid}/fertsite/{type}/log GET /targets/{id}/lines/{lid}/fertsite/{type}/{pos}/log {id} is the serial number of controller {lid} is the line id (1...) {sid} is the site id (1...) {type} is type of object (fert, fertmeter, booster, agitator) {pos} is the orderly number of object in position
Parameters	same as above
Response JSON	same as above
Examples	*gets data of booster of local fert site in line 4* GET /targets/123/lines/4/fertsite/booster/log? from=X&until=Y&otype=**DIGITAL**
	gets data of fertmeter 3 in central fert site 2 GET /targets/123/fertsites/2/fertmeters/3/log? from=X&until=Y&otype=**ANALOG**
	gets fertilizer accumulation of injector #3 of central site #1 GET /targets/123/fertsites/1/ferts/3/log? from=X&until=Y&otype=**FERT_ACC**
	gets data local fert site in line 3 GET /targets/123/lines/3/fertsite/log?from=X&until=Y&otype=**DIGITAL**

6.23.5 Raw data queries (back flushing)

These calls for raw data are specifically mapped for objects in central and local filtration systems, such as sites, filter valves, DP sensors, etc.

URI	**Central filtration calls:** GET /targets/{id}/filtersites/{sid}/{type}/log GET /targets/{id}/filtersites/{sid}/{type}/{pos}/log **Local filtration calls:** GET /targets/{id}/lines/{lid}/filtersite/log GET /targets/{id}/lines/{lid}/filtersite/{type}/log GET /targets/{id}/lines/{lid}/filtersite/{type}/{pos}/log {id} is the serial number of controller {lid} is the line id (1...) {sid} is the site id (1...) {type} is type of object (filter valves, dp, down stream valves) {pos} is the orderly number of object in position
Parameters	same as above
Response JSON	same as above
Examples	*gets data of DP sensor in local filter site of line 4* GET /targets/123/lines/4/filtersite/dp/log? from=X&until=Y&otype=DIGITAL
	gets data of filter valve 3 in central filter site 2 GET /targets/123/filtersites/2/filter/3/log? from=X&until=Y&otype=DIGITAL
	gets data local filter site in line 3 GET /targets/123/lines/3/filtersite/log? from=X&until=Y&otype=DIGITAL

6.23.6 Water consumption queries

The following calls perform water consumption query for a single valve or water meter. Query is based on required time range and rate. Note how objects are referenced with URI.

The response is a single data series where each observation contains time range and water consumption volume (or time) in this range. Number of observation will be equal query time range divided by rate.

URI	**For getting valve water consumption:** GET /targets/{id}/lines/{lid}/valves/{vid}/wc **For getting local water meter consumption:** GET /targets/{id}/lines/{lid}/watermeter/wc **For getting free water meter consumption** GET /targets/{id}/watermeters/{wid}/wc **Where:** {id} is the serial number of controller {lid} is line number {vid} valve position in line {wid} free water meter number	
Parameters	from	*epoch time (UTC, milliseconds)*
	until	*epoch time (UTC, milliseconds)*
	rate	*hourly, daily, weekly, monthly*
	volume	***optional**, 'true' for volume (default), 'false' for time*
Response	[{	*array of arrays of consumptions*
	from: number	*epoch time (UTC, milliseconds)*
	until: number	*epoch time (UTC, milliseconds)*
	value: number	*water consumption volume at this time range*
	valuePerArea: number	*water consumption volume per area unit at this time range*
	},...]	

6.23.7 Water consumption batch queries

The following call performs water consumptions query for arbitrary number of valves. Similar to a single object requests, those queries produce the same data series but aggregated per individual object requested. Note the format specifying the requested valves.

URI	**For getting water consumptions of several valves:** GET /targets/{id}/wc/valves **Where:** {id} is the serial number of controller	
Parameters	vids	***optional**, comma separated list of valves. Accepts the format of **line.valve** (1.1,1.2,..), or **IDD** format (11:1, 11:2, 11:3,..)*
	from	*epoch time (UTC, milliseconds)*
	until	*epoch time (UTC, milliseconds)*
	rate	***optional**, hourly, daily, weekly, monthly (default is daily)*
	volume	***optional**, 'true' for volume (default), 'false' for time*
Response	{ uid: [{	*collection of arrays each corresponds to the specific valve by its uid*
	from: number	*epoch time (UTC, milliseconds)*
	until: number	*epoch time (UTC, milliseconds)*
	value: number	*water consumption volume at this time range*
	valuePerArea: number	*water consumption volume per area unit at this time range*
	},...],... }	
Examples	*gets hourly water consumption of valves 1,2 and 3 in line 1* GET /targets/123/wc/valves? vids=1.1,1.2,1.3&from=X&until=Y&rate=hourly	

Sometimes it is convenient to obtain water consumptions of entire irrigation line. This will include local water meter (if present) and all valves of the line. Note that if requested line has local water meter, then its consumptions will always be the first array entry in the returned results, otherwise first entry will be the first valve.

URI	**For getting water consumptions of entire irrigation line:** GET /targets/{id}/lines/{lid}/wc **Where:** {id} is the serial number of controller {lid} is the line id	
Parameters	from	*epoch time (UTC, milliseconds)*
	until	*epoch time (UTC, milliseconds)*
	rate	***optional**, hourly, daily, weekly, monthly (default is daily)*
	volume	***optional**, 'true' for volume (default), 'false' for time*
Response	{ uid: [{	*collection of arrays each corresponds to the specific object by its uid*
	from: number	*epoch time (UTC, milliseconds)*
	until: number	*epoch time (UTC, milliseconds)*
	value: number	*water consumption volume at this time range*
	valuePerArea: number	*water consumption volume per area unit at this time range*
	},...],... }	
Examples	*gets hourly water consumption of line 5 objects* GET /targets/123/line/5/wc?from=X&until=Y&rate=hourly	

It is also possible to obtain water consumptions of multiple water meters per type. These controllers currently support three types of water meters - installed with water sources, installed in irrigation lines, and free (installed in arbitrary locations). You will specify the water meter type as well as the list of id's within this type namespace, see examples below.
Please Note to get the consumption of virtual water meters you will need to use a separate endpoint with the same parameter. Unfortunately, we are not able to create a united request for all types of water meter.

URI	**For getting consumptions of arbitrary water meters:** GET /targets/{id}/wc/watermeters GET /targets/{id}/wc/virtualwatermeters **Where:** {id} is the serial number of controller	
Parameters	type	***optional**, type of water meters – source, line, free*
	ids	***optional**, comma separated list of water meter ids within specified type*
	from	*epoch time (UTC, milliseconds)*
	until	*epoch time (UTC, milliseconds)*
	rate	***optional**,* hourly, daily, weekly, monthly (default is daily)
	volume	***optional**, 'true' for volume (default), 'false' for time*
Response	{ uid: [{	*collection of arrays each corresponds to the specific water meter by its uid*
	from: number	*epoch time (UTC, milliseconds)*
	until: number	*epoch time (UTC, milliseconds)*
	value: number	*water consumption volume at this time range*
	valuePerArea: number	*water consumption volume per area unit at this time range*
	},...],... }	

Examples	*gets daily consumption of all water meters in the system* GET /targets/123/wc/watermeters?from=X&until=Y *gets daily consumption of water meters in lines 3,4,5* GET /targets/123/wc/watermeters? type=line&ids=3,4,5&from=X&until=Y&rate=hourly *gets daily consumption of all virtual water meters in the system with rate weekly* GET /targets/123/wc/ virtualwatermeters?from=X&until=Y&rate=weekly

6.23.8 Fertilizer consumption queries

The following set of queries gets fertilizer consumptions for a single valve, fert injector or fert meter. Queries are based on time range and rate. If NPK is used by given controller then it can optionally be requested as well. By default, NPK will not be included.

URI	**For fertilizer consumptions of a single valve:** GET /targets/{id}/lines/{lid}/valves/{vid}/fc **For fertilizer consumptions of local fert injector:** GET /targets/{id}/lines/{lid}/fertsite/ferts/{fid}/fc **For fertilizer consumptions of local fert meter:** GET /targets/{id}/lines/{lid}/fertsite/fms/{fid}/fc **For fertilizer consumptions of central fert injector:** GET /targets/{id}/fertsites/{sid}/ferts/{fid}/fc **For fertilizer consumptions of central fert meter:** GET /targets/{id}/fertsites/{sid}/fms/{fid}/fc **Where:** {id} serial number of the controller {lid} line number {sid} site number {vid} valve position in line {fid} fert injector (or meter) position in site	
Parameters	from	*epoch time (UTC, milliseconds)*
	until	*epoch time (UTC, milliseconds)*
	rate	*hourly, daily, weekly, monthly (default is daily)*
	npk	**optional**, true if NPK is needed

Depending on the object of interest, the response will contain one ore more data series. For example, fertilizer consumptions of a valve will contain data series for each fert injector connected to requested valve. Fertilizer consumptions of fert meter will include single data series of fert injector to which fert meter is attached.

Here is a simple naming convention for data series names:

`lf.1.1`	*volume of fertilizer from first injector in first irrigation **line***
`cf.2.3`	*volume of fertilizer from third injector in second **central** site*
`lf.3.1.p`	*volume of Phosphorous component from first injector in third line*
`cf.2.1.n`	*volume of Natrium component from first injector in second site*

Here is an example of fertilizer consumptions for a single irrigation valve. Since each valve may be connected to several fertilizers, the response will contain data series for each injector separately. Plus, if requested, the NPK data (not shown here).

```
[
  {
    "lf.1.1": [
      {
        "from": 1478988000000,          // epoch time (UTC, milliseconds)
        "until": 1479074399999,         // epoch time (UTC, milliseconds)
        "value": 10,                    // consumption of fert in this time range
        "valuePerArea": 10              // used when area coverage is configured
      }
    ]
  },
  {
    "lf.1.2": [
      {
        "from": 1478988000000,
        "until": 1479074399999,
        "value": 13,
        "valuePerArea": 13
      }
    ]
  },
...
]
```

Number of observations in each data series will be equal query time range divided by rate.

6.23.9 Fertilizer consumption batch queries

The following are the fertilizer consumption queries targeted at all fertilizer injectors (or meters) in the system. The structure of responses produced by these calls are identical to the calls for individual objects as illustrated in previous paragraph. In case of valve per injector consumptions the response will be aggregated by the valves but the underlying structure is the same.

URI	**Fertilizer consumptions of all valves** GET /targets/{id}/fc/valves **Fertilizer consumptions of all fert injectors** GET /targets/{id}/fc/ferts **Fertilizer consumptions of all fertmeters** GET /targets/{id}/fc/fertmeter **Where:** {id} serial number of the controller	
Parameters	vids	***optional***, *comma separated list of valves. Accepts the format of* ***line.valve*** *(1.1,1.2,..), or* ***IDD*** *format (11:1, 11:2, 11:3,..), used only for valves fert consumptions*
	from	*epoch time (UTC, milliseconds)*
	until	*epoch time (UTC, milliseconds)*
	rate	***optional***, *hourly, daily, weekly, monthly (default is daily)*
	npk	**optional**, true if NPK is needed
	uid	**optional**, true will add "uid" parameter to each fert array inside of the response

6.23.10 Analysis library queries

Analysis library is a storage holding seasons and fertilizer definitions. This information is available per specific fertilizer injector at a given time point:

URI	**Fertilizer recipe for local site injector** GET /targets/{id}/lines/{lid}/fertsite/ferts/{fid}/recipe **Fertilizer recipe for central site injectors** GET /fertsites/{sid}/ferts/{fid}/recipe **Where:** {id} serial number of the controller {lid} the irrigation line id {fid} the fertilizer injector position in site {sid} the fertilizer site id	
Parameters	**time**	*epoch time (UTC, milliseconds) for the season. Can be any time point within the required season where fertilize recipe is applied.*
Response JSON	Single entry of fertilizer recipe, see appendix for the reference	
Examples	*the following will get the fertilizer recipe of second injector from first irrigation line for the time point of 1 Jan 2020:* GET /targets/123/lines/1/fertsite/fert/2/recipe?time=1577836800000	

It is also possible to obtain collection of all fertilizer recipes used by particular irrigation controller:

URI	**All fertilizer recipes for specific controller** GET /targets/{id}/analib/recipes **Where:** {id} serial number of the controller	
Parameter	uid	**optional**, true will add "uid" parameter to each fert array inside of the response
Response JSON	Collection of fertilizer recipes, see appendix for the reference	

6.23.11 Event log queries

Events raised by irrigation controller during its operation can be queried by specifying the time range of interest. As of this writing the event log is kept for at least 3 months.

URI	GET /targets/{id}/eventlog **Where:** {id} serial number of the controller	
Parameters	from	*epoch time (UTC, milliseconds)*
	until	*epoch time (UTC, milliseconds)*
Response JSON	[{	*array of objects*
	time:	*the time when event took place, UTC, ms*
	context:	*context parameter (optional)*
	subcontext:	*subcontext parameter (optional)*
	message:	*message in the language of the caller*
	},...]	

6.24 GIS

SPOT server is equipped with database holding geospatial information of controller objects.

6.24.1 Object locations

The caller will get a collection of descriptors associated with a physical component in a controller image (e.g. Valve, Water meter, Plot, etc). The descriptor holds an information about controller type, its serial number, object identifier, custom properties and geometry (point, polygon, etc).

The “location” attribute of the structure below describes the geometry and geographical position in accordance with GeoJSON specification. For example, a “valve” is described as a “Point” with a single coordinate (longitude and latitude) while “plot” will be described as “Polygon” with coordinates for each vertex.

URI	`GET /targets/{id}/locations` `{id} is the serial number of controller`	
Response JSON	`[` `  {`	*array of location objects, each entry maps uniquely to a physical object in irrigation controller*
	`dtp: enum`	*device (controller) type* *1 – dream* *4 – sapir*
	`tid: number`	*serial number of the controller*
	`otp: enum`	*object type, see Appendix for details*
	`oid: number`	*object identifier within type namespace*
	`pps: object`	*custom properties, a name-value pairs, optional*
	`location: object`	*GeoJSON geometry object, depends on the underlying object, more details can be found here.*
	`  },...` `]`	

6.25 Analog values

URI	GET /targets/{id}/avalues GET /targets/{id}/avalues/{id}		
Description	Gets individual object or all analog values as a batch		
Response	avalues: {		
	uid:	string	*unique object id*
	id:	number	*position of RTU in the interface card map*
	name:	string	*user specified name or alias*
	state:	enum	*0 - offline* *1 - online* *2 - communication error* *3 - not connected*
	online:	boolean	*indicates that data is live or cached*
	card:	number	*index of interface card*
	rtu:	number	*index of RTU in the card*
	plugin:	number	*index of the plugin card on the RTU*
	mbu:	number	*index of the MBU in the system*
	output:	number	*output number used*
	type:	enum	*type of sensor (see sensors types table)*
	units:	enum	*measurement units (see units types table)*
	value:	number	*current value*
	minLimit:	number	*minimal limit of the analog value*
	maxLimit:	number	*maximal limit of the analog value*
	hysteresis:	number	*hysteresis of the analog value*
	}		

7 Appendixes

7.1 URL examples

	Controller related calls
Get list of my controller structures	GET /api/targets
Get structure of controller #1	GET /api/targets/1/
List states of my controllers	GET /api/targets?filter=state
List serial numbers of my controllers	GET /api/targets?filter=serial
Get state of controller 1 battery	GET /targets/1/battery
Get alarm 2 state of controller 1	GET /targets/1/alarms/2
	Main valves
List all main valves of controller 1	GET /targets/1/mainvalves
Get main valve 2 of controller 1	GET /targets/1/mainvalves/2
	Water sources
List all water sources of controller 1	GET /targets/1/sources
Get water source B of controller 1	GET /targets/1/sources/2
Get a water meter of water source B of controller 1	GET /targets/1/sources/2/watermeter
List all pumps of water source B in controller 1	GET /targets/1/sources/2/pumps
Get third pump of water source B in controller 1	GET /targets/1/sources/2/pumps/3
	Free water meters
List all free water meters of controller 1	GET /targets/1/watermeters
Get third free water meter of controller 1	GET /targets/1/watermeters/3
	Virtual water meters
List all virtual water meters of controller 1	GET /targets/1/virtual
Get third virtual water meter of controller 1	GET /targets/1/virtual/3

	Analog sensors
List all analog sensors of controller 1	GET /targets/1/sensors
Get third analog sensor of controller 1	GET /targets/1/sensors/3
	Central Fertilization
Get all fertilizer injectors in a flat list	GET /targets/{id}/ferts
Get all fert meters in a flat list	GET /targets/{id}/fertmeters
List central fertilization sites of controller 1	GET /targets/1/fertsites
Get fertilization site 2 of controller 1	GET /targets/1/fertsites/2
Get booster of central fertilization site 2 of controller 1	GET /targets/1/fertsites/2/booster
Get ECPH controller of central fertilization site 2 of controller 1	GET /targets/1/fertsites/2/ecph
List all injectors of central fertilization site 2 of controller 1	GET /targets/1/fertsites/2/ferts
Get third injector of central fertilization site 2 of controller 1	GET /targets/1/fertsites/2/ferts/3
	Central Filtration
List central filtration sites of controller 1	GET /targets/1/filtersites
Get filtration site 2 of controller 1	GET /targets/1/filtersites/2
List all filter valves of central filtration site 2 of controller 1	GET /targets/1/filtersites/2/filters
Get third filter valve of central filtration site 2 of controller 1	GET /targets/1/filtersites/2/filters/3
	Lines
List all irrigation lines of controller 1	GET /targets/1/lines
Get irrigation line 2 of controller 1	GET /targets/1/lines/2
Get line 2 water meter in controller 1	GET /targets/1/lines/2/watermeter
Get line 2 pressure meter in controller 1	GET /targets/1/lines/2/pressmeter

List line 2 valves in controller 1	GET /targets/1/lines/2/valves
Get valve 3 in line 2 of controller 1	GET /targets/1/lines/2/valves/3
Get main valve connected to line 2 in controller 1	GET /targets/1/lines/2/mainvalve
Get local fertilization site of line 2 in controller 1	GET /targets/1/line/2/fertsite
List all fert injectors of local fertilization site in line 2 of controller 1	GET /targets/1/line/2/fertsite/ferts
Get third injector of local fertilization site in line 2 of controller 1	GET /targets/1/line/2/fertsite/ferts/3
Get local filtration site of line 2 in controller 1	GET /targets/1/line/2/filtersite
List all filter valves of local filtration site in line 2 of controller 1	GET /targets/1/line/2/filtersite/filters
Get third filter valve of local filtration site in line 2 of controller 1	GET /targets/1/line/2/filtersite/filter/3
	Programs
List all programs in controller 1	GET /targets/1/programs
Get program 2 of controller 1	GET /targets/1/programs/2
	Interfaces
List all interface cards in controller 1	GET /targets/1/interfaces
Get interface card 2 of controller 1	GET /targets/1/interfaces/2
List RTU's of interface card 2 in controller 1	GET /targets/1/interfaces/2/rtus
Get third RTU connected to interface 2 in controller 1	GET /targets/1/interfaces/2/rtus/3
	Conditions
List all conditions defined in controller 1	GET /targets/1/conditions
Get condition 2 in controller 1	GET /targets/1/conditions/2
	Contacts
List all contacts in controller 1	GET /targets/1/contacts
Get contact 2 in controller 1	GET /targets/1/contacts/2

	Weather station
Get a weather station of controller 1	GET /targets/1/weather
	Direct access to objects with uid
Get the first valve in controller with serial number 1	GET /targets/1/idd/11/1
Get the fifth filter in controller with serial number 1	GET /targets/1/idd/13/5
Get tenth analog sensor	GET /targets/1/idd/31/10
	Query observations
Get analog observations of sensor #4 in time range X - Y in controller 1	GET /targets/1/sensors/4/log? from=X&until=Y&otype=ANALOG
Get flow observations of free water meter #2 in time range X - Y in controller 1	GET /targets/1/watermeters/2/log? from=X&until=Y&otype=ANALOG
Get states of condition #3 in time range X - Y in controller 1	GET /targets/1/conditions/3/log? from=X&until=Y&otype=DIGITAL
Get states of contact #5 in time range X - Y in controller 1	GET /targets/1/contacts/5/log? from=X&until=Y&otype=DIGITAL
Get battery voltage history in time range X - Y in controller 1	GET /targets/1/battery/1/log? from=X&until=Y&otype=ANALOG
Gets program 2 start/stop logs in time range X - Y in controller 1	GET /targets/1/programs/2/log? from=X&until=Y&otype=DIGITAL
	GIS call
Get all GIS locations of controller with serial number 1	GET /targets/1/locations
	MBUs call
Get list of the all MBUs defined in the controller	GET /targets/1/mbus

7.2 Response codes

0	OK
101	INVALID_FORMAT
102	INVALID_MODE
103	INVALID_DATA
104	INVALID_INDEX
105	INVALID_ARGUMENT
106	INVALID_COMMAND
107	INVALID_STATUS
110	INVALID_PARAM
111	INVALID_OPERATION
1000	ACCESS_DENIED
1001	ACCESS_CREDENITIALS
1002	ACCESS_MULTIPLE
1003	ACCESS_AUTHORIZATION
1004	ACCESS_COOKIE
1005	ACCESS_NOADMIN
1100	INVALID_RESPONSE
1101	NO_TARGET_CONNECTION
1102	NO_SERVER_CONNECTION
1103	SERVER_CONNECTION_REFUSED
1104	PERMANENT_FAILURE

7.3 Dream (Sapir) alarm types

Index	Description
1	Device is frozen
2	Line is frozen
3	RTU 2W line overload
4	Low battery
5	Low pressure
6	Output short circuit
8	Communication with peripherals
9	RTU communication problem
10	No AC
11	Illegal peripheral card
12	Water leakage
13	Fertilizer leakage
14	DP looping
15	Lack of fertilizer
16	Water high flow
17	Water low flow
18	Repeated flow problem
19	Net protection
20	RTU low battery
21	pH failure
22	EC failure
24	EC or pH sensors data mismatch
32	Rain delay
33	Frost protection
34	Unresponsive valve problem

35	Solenoid is disconnected
36	Debounce problem
37	Improper peripheral module configuration

7.4 Object types

Enum	Description
1	Controller
4	Irrigation line
5	Main valve
6	Water meter
7	Water source
8	Water pump
11	Valve
12	Filter site
13	Filter valve
14	Downstream valve
15	Differential pressure sensor
16	Fertilization site
17	Fertilization valve
18	Fert meter
19	Booster
20	Agitator
21	ECPH controller
22	Pressure meter
25	Virtual water meter
26	Interface card
28	Satellite
29	Alarm output
31	Analog sensor
34	Davis weather station
35	Contact
37	Battery
45	RTU
49	Analog value holder

51	Plugin card
52	ModBUS device

7.5 Observation types

Note that either of the formats (IDD number and Enumerator) can be used with requests.

IDD	Enum	Description
1	SYSTEM_EVENT	System events generated by controller
2	ANALOG	Analog values (sensors, meters, batteries, etc)
3	DIGITAL	Digital values (opening, closing of outputs, programs)
4	WATER_ACC	Water volume accumulations
5	FERT_ACC	Fertilizer volume accumulations
6	RAD_ACC	Radiation accumulations
7	COMM_RROR	RTU communication errors
8	WATER_TIME_ACC	Water time accumulations
9	PROGRAM_LOG	Program irrigation log

7.6 Analog sensor types

Enum	Description
0	Not used
1	Sensor measuring temperature
2	Sensor measuring humidity
3	Tensiometer
4	Water pressure sensors
5	Sensor measuring sun radiation
6	Sensor measuring EC
7	Sensor measuring PH
8	Sensor measuring flow of water
9	Phyto
10	Wind speed detector
11	Wind direction detector
12	Dendrometer
13	Atmospheric pressure sensor
14	Daily rain sensor
15	Rain rate sensor
16	UV radiation sensor
17	Evapotranspiration sensor
18	Dew point sensor
19	Differential pressure sensor
20	Pressure sensor
21	Stem diameter
22	Fruit diameter
23	Leaf temperature
24	Soil moisture
25	VWC MAS-1
26	Soil temperature
27	Dielectric permittivity
28	Par

29	Leaf wetness
30	Current engine hours
31	Current RPM
32	Voltage
33	Current oil pressure
34	Current engine temperature
35	Discharge pressure
36	System level
37	Precipitation
38	Plant length
39	VWC_Terros10
42	CO2 level
43	Power
44	Charging
45	Concentration
46	Volume
47	Salinity
48	Total dissolved solids (TDS)
49	Chill accumulator
50	Frequency

7.7 Condition types

Type	Description
0	UNDEFINED
1	PROGRAM_RUNNING
2	PROGRAM_NOT_RUNNING
3	PROGRAM_STARTING
4	PROGRAM_ENDING
5	CONTACT_OPENED
6	CONTACT_CLOSED
7	CONTACT_OPENING
8	CONTACT_CLOSING
9	SATELLITE_OPENED
10	SATELLITE_CLOSED
11	SATELLITE_OPENING
12	SATELLITE_CLOSING
13	WM_FLOW_HIGHER
14	WM_FLOW_LOWER
15	VM_FLOW_HIGHER
16	VM_FLOW_LOWER
17	SENSOR_HIGHER
18	SENSOR_LOWER
19	COMBINED_TRUE
20	COMBINED_FALSE

7.8 Analog sensor units

Enum	Description
0	not specified
1	Hg inch
2	hg mm
3	hPa
4	millibar
5	kPa
6	%
7	celsius
8	fahrenheit
9	inch
10	mm
11	m/h
12	km/h
13	m/s
14	kt
15	Degrees (grad)
16	W/m2
17	UV index
18	med
19	inch/h
20	mm/h
21	Index (No units)
22	centibar
23	m/(h_2o)
24	psi
25	atm

26	par
27	µE
28	lux
29	mSiemens/cm
30	m/s
31	km/h
32	ft/s
33	m^3/h
34	g/min
35	mSiemens
36	pH
37	mA
38	V
39	µmol m-2 s-1
40	micron
41	rotations per minute
42	feet
43	hours
44	bar
45	meter
46	gallon per hour
47	volumetric ion concentration
48	litre per hour
49	part per million
50	kW
51	Mg/l
52	m3
53	L

54	ppt (parts per thousand)
55	mV
56	µSiemens/cm
57	Herz

7.9 Fertilizer recipes

The following structure describes a fertilizer recipe used in server Analysis Library to track different kind of fertilizers used at different seasons.

```
{
    "name": "My recipe",                        // fertilizer name
    "gravity": 0.1,                             // fertilizer parameters..
    "npercent": 0.2,
    "ppercent": 0.3,
    "kpercent": 0.1,
    "ncoef": 1.0,
    "pcoef": 0.44,
    "kcoef": 0.83,
    "delution": 100.0,
    "m1": 0.1,
    "m2": 0.1,
    "m3": 0.2,
    "m4": 0.2,
    "m5": 0.3,
    "m6": 0.4,
    "items": [                                  // linked fert injectors
        {
            "uid": "17:1",                      // if parameter is activated
            "fertilizer": "lf.1.1",             // fert injector reference
            "seasonName": "Spring",             // season used with injector
            "seasonStartTime": 1583020800000,   // season range (epoch time)
            "seasonEndTime": 1590883200
        },
        {
            "uid": "17:1",                      // if parameter is activated
            "fertilizer": "cf.2.5",             // fert injector reference
            "seasonName": "Summer",
            "seasonStartTime": 1590969600000,
            "seasonEndTime": 1598918400000
        }
    ]
}
```

8 Requests responses filtering

In the new version of API, we added a new option which will allow users to get only the data which they need. This option is called filtering. Almost to each GET request user can add the filtering command and filtering parameters. The response will be filtered according to your request filtering parameters and you will get the data that you need only.

The rules of the request are the following: first you need to write the name of the container which are the same as chapter names of this document starting from 6.1 and finishing with 6.20. Otherwise, you can analyze the tree structure of the full controller image request response and understand which container names and parameters do you really need.

Please be aware that if you have the same parameter names in different containers and are selected to show it in one of the containers – you will get this parameter in all containers in the response.

URI	GET /targets/{id}/info
Description	Gets info container of the controller
Response	{ "serial": {id}, "name": "SAPIR Name", "app": 4, "version": "v1.1.720", "tzOffset": 180, "metric": 0 }
URI	GET /targets/{id}/info?filter=serial|version
Description	Gets info container of the controller only with parameters “serial” and “version”
Response	{ "serial": {id}, "version": "v1.1.720" }

URI	GET /targets/{id}/valves?filter=uid|name|state|acc
Description	Gets list of all valves of the controller with parameters “uid”, “name”, “state” and “acc”.
Response	[{ "uid": "11:1", "name": "Valve_Test_1", "state": 7, "acc": 19123.0 }, {

```
            "uid": "11:2",
            "name": "test name",
            "state": 7,
            "acc": 0.036
        },
        {
            "uid": "11:3",
            "name": "Valve 1.3",
            "state": 1,
            "acc": 0.62
        },
        {
            "uid": "11:4",
            "name": "Valve 1.4",
            "state": 0,
            "acc": 0.0
        },
……..
]
```


When you need to get data from the several containers you can filter the full controller image. In this case you will need to add to the filter container names as well. Like we have it here: *?filter=container1|parameter1|parameter2|parameter3|container2|parameter4|parameter5|…*
It is possible to write the container's names first and only then start with the list of needed parameters.

URI	GET /targets/{id}/?filter=lines|waterMeter|sensors|valves|uid|name|value|type|unit|acc|accTime|area|flow
Description	Gets the following parameters: "uid|name|value|type|unit|acc|accTime|area|flow" from the following containers: "lines|waterMeter|sensors|valves".
Response	{ "sensors": [{ "uid": "31:1", "name": "Analog sensor 1", "type": 13, "value": 0.0 }, { "uid": "31:2", "name": "Analog sensor 2", "type": 12, "value": 0.0 }, { "uid": "31:3", "name": "Analog sensor 3", "type": 0, "value": 0.0 }, ……..

```
        {
            "uid": "31:12",
            "name": "Virtual Daily Rain",
            "type": 14,
            "value": 0.0
        }
    ],
    "lines": [
        {
            "uid": "4:1",
            "name": "Irrigation line 1",
            "waterMeter": {
                "uid": "6:1",
                "name": "Line 1 water meter",
                "type": 0,
                "flow": 0.0,
                "acc": 0.0
            },
            "valves": [
                {
                    "uid": "11:1",
                    "name": "Valve 1.1",
                    "area": 6499.0,
                    "acc": 2147483.0,
                    "accTime": 100180300
                },
                {
                    "uid": "11:2",
                    "name": "Valve 1.2",
                    "area": 1.1,
                    "acc": 1.0,
                    "accTime": 166560
                },
                {
                    "uid": "11:3",
                    "name": "Valve 1.3",
                    "area": 1.0,
                    "acc": 0.0,
                    "accTime": 8778020
                },
                {
                    "uid": "11:4",
                    "name": "Valve 1.4",
                    "area": 1.0,
                    "acc": 705032.7,
                    "accTime": 8778020
                },
                {
                    "uid": "11:5",
                    "name": "Valve 1.5",
                    "area": 5000.0,
                    "acc": 705032.7,
                    "accTime": 422663
                },
                {
```

```
                        "uid": "11:6",
                        "name": "Valve 1.6",
                        "area": 1.0,
                        "acc": 1705032.8,
                        "accTime": 422635
                    },
                    {
                        "uid": "11:7",
                        "name": "Valve 1.7",
                        "area": 1.0,
                        "acc": 0.0,
                        "accTime": 860897
                    },
                    {
                        "uid": "11:8",
                        "name": "Valve 1.8",
                        "area": 1.0,
                        "acc": 1410065.4,
                        "accTime": 0
                    }
                ]
            }
        ]
}
```